玩转数据 120 题：tidyverse 版（2026）

Tidyverse 入门练习

张敬信

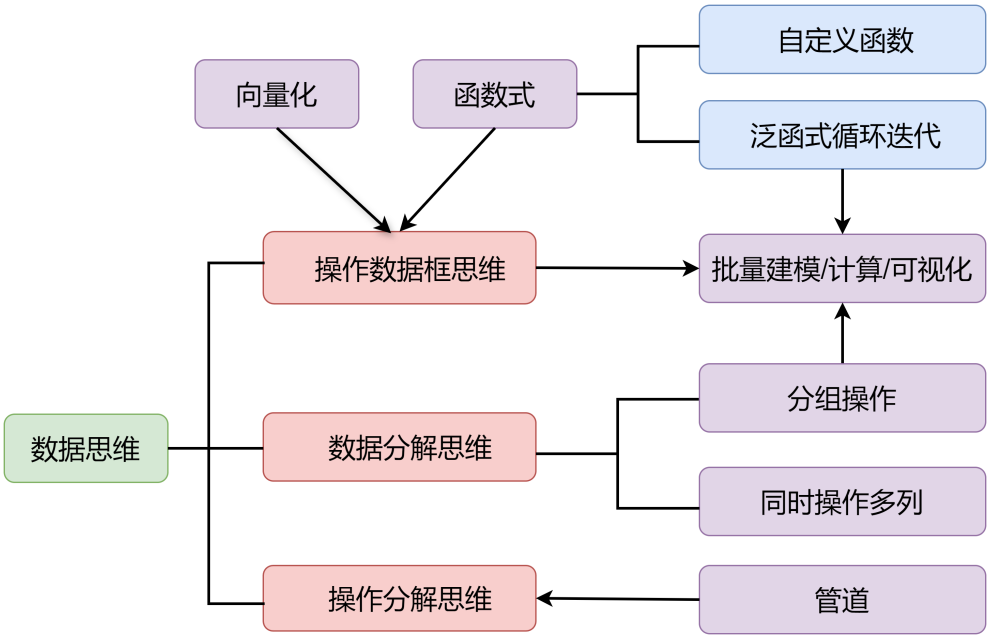
2026-10-13

关于作者：

- 张敬信，哈尔滨商业大学，数学与应用数学，副教授
- 热爱学习，热爱编程，热爱 R 语言
- 我的代表作《R 语言编程：基于 tidyverse》已于 2023 年 2 月正式上市，荣登 2023 年度人邮畅销新书榜第 5 名，异步社区、京东、天猫、当当等各大平台有售：



本书在系统梳理 tidyverse 数据操作逻辑的基础上，创造性地提出了数据编程思想，可以说是解决一切数据操作问题的指导性思想：



该思维导图的快速解读，可参阅优雅数据编程思维（实例阐述）。

- 本书免费学习资源：课件和鲸在线版（可调试代码），知乎交流平台，欢迎您的阅读品鉴、交流讨论！
- 本书的 QQ 读者 1 群、2 群：875664831、222427909, 交流、答疑，欢迎您的加入！
- 我的下一部著作《R 机器学习：基于 mlr3verse》，即将上市，也可以关注我的 R 基础、R 机器学习培训班！

玩转数据 120 题来自刘早起的 *Pandas 进阶修炼 120 题*，涵盖了数据处理、计算、可视化等常用操作，希望通过 120 道精心挑选的习题吃透 pandas.

后来，中山大学博士陈熹提供了 R 语言版本。我[1][2][3]再来个更能体现 R 语言最新技术的 tidyverse 版本。

关于**更新版**：感谢@鼠大米对部分解法不够 tidyverse 的题目，提供了新解法 (再加上我稍微修正)，主要是加入更好用的新函数 slice_*()。2023 年，部分题目又做了一些改进。2024 年，部分题目又做了一些改进，同时改用 Quarto。2026 年，管道符从%>% 换成 |>，改用 pdf-engine: lualatex。

- 先加载包：

```
library(tidyverse)
```

1 Part I 入门

1.1 题目 1（创建数据框）：将下面的字典创建为 DataFrame

data = {"grammer": ["Python","C","Java","GO",np.nan,"SQL","PHP","Python"],
"score": [1,2,np.nan,4,5,6,7,10]}

难度：★

代码及运行结果：

```
df = tibble(
  grammer = c("Python","C","Java","GO", NA,"SQL","PHP","Python"),
  score = c(1,2,NA,4,5,6,7,10))
df
```

```
# A tibble: 8 x 2
  grammer score
  <chr>   <dbl>
1 Python      1
```

[1]我的 Gitee: https://gitee.com/zhjx19

[2]我的知乎: https://www.zhihu.com/people/huc_zhangjingxin

[3]我的和鲸项目（包含本文件的在线可调试版）：https://www.heywhale.com/home/user/profile/5d4a42a636e903002c0e0c66/project

```
2 C           2
3 Java       NA
4 GO          4
5 <NA>        5
6 SQL         6
7 PHP         7
8 Python     10
```

- **补充**：按行录入式创建数据框

```
df = tribble(
  ~ grammer, ~ score,
  "Python",  1,
  "C",       2,
  "Java",    NA,
  "GO",      4,
  NA,        5,
  "SQL",     6,
  "PHP",     7,
  "Python",  10)
```

1.2 问题 2（筛选行）：提取含有字符串"Python"的行

难度：★

代码及运行结果：

```
df |>
  filter(grammer == "Python")
```

```
# A tibble: 2 x 2
  grammer score
  <chr>   <dbl>
1 Python      1
2 Python     10
```

1.3 题目 3（查看列名）：输出 df 的所有列名

难度：⋆

代码及运行结果：

```
names(df)
```

```
[1] "grammer" "score"
```

1.4 题目 4（修改列名）：修改第 2 列列名为”popularity”

难度：⋆⋆

代码及运行结果：

```
df = df |>
  rename(popularity = score)
df
```

```
# A tibble: 8 x 2
  grammer popularity
  <chr>        <dbl>
1 Python           1
2 C                2
3 Java            NA
4 GO               4
5 <NA>             5
6 SQL              6
7 PHP              7
8 Python          10
```

1.5 题目 5（统计频数）：统计 grammer 列中每种编程语言出现的次数

难度：★★

代码及运行结果：

```
df |>
  count(grammer)
```

```
# A tibble: 7 x 2
  grammer     n
  <chr>   <int>
1 C           1
2 GO          1
3 Java        1
4 PHP         1
5 Python      2
6 SQL         1
7 <NA>        1
```

1.6 题目 6（缺失值处理）：将空值用上下值的平均值填充

难度：★★★

代码及运行结果：

```
df = df |>
  mutate(popularity = zoo::na.approx(popularity))
df
```

```
# A tibble: 8 x 2
  grammer popularity
  <chr>        <dbl>
1 Python           1
2 C                2
3 Java             3
```

```
4 GO                4
5 <NA>              5
6 SQL               6
7 PHP               7
8 Python           10
```

注：tidyr 包提供了 fill() 函数，可以用前值或后值插补缺失值。

1.7 题目 7（筛选行）：提取 popularity 列中值大于 3 的行

难度：★★

代码及运行结果：

```
df |>
  filter(popularity > 3)
```

```
# A tibble: 5 x 2
  grammer popularity
  <chr>        <dbl>
1 GO               4
2 <NA>             5
3 SQL              6
4 PHP              7
5 Python          10
```

1.8 题目 8（数据去重）：按 grammer 列进行去重

难度：★★

代码及运行结果：

```
df |>
  distinct(grammer, .keep_all = TRUE)
```

```
# A tibble: 7 x 2
  grammer popularity
  <chr>        <dbl>
1 Python           1
2 C                2
3 Java             3
4 GO               4
5 <NA>             5
6 SQL              6
7 PHP              7
```

1.9 题目 9（数据计算）：计算 popularity 列平均值

难度：★★

代码及运行结果：

```
df |>
  summarise(avg = mean(popularity))
```

```
# A tibble: 1 x 1
    avg
  <dbl>
1  4.75
```

1.10 题目 10（格式转换）：将 grammer 列转换为向量

难度：★

代码及运行结果：

```
df |>
  pull(grammer)    # 或者 df$grammer
```

```
[1] "Python" "C"      "Java"   "GO"     NA       "SQL"    "PHP"    "Python"
```

注：R 从数据框中提取出来就是字符向量。

1.11 题目 11（数据保存）：将数据框保存为 Excel 文件

难度：★★

代码及运行结果：

```
library(writexl)
write_xlsx(df, "data/filename.xlsx")
```

1.12 题目 12（数据查看）：查看数据的行数列数

难度：★

代码及运行结果：

```
dim(df)
```

```
[1] 8 2
```

1.13 题目 13（筛选行）：提取 popularity 列值大于 3 小于 7 的行

难度：★★

代码及运行结果：

```
df |>
  filter(popularity > 3, popularity < 7)
```

```
# A tibble: 3 x 2
  grammer popularity
  <chr>        <dbl>
1 GO               4
2 <NA>             5
3 SQL              6
```

1.14 题目 14（调整列位置）：交互两列的位置

难度：★★

代码及运行结果：

```
df |>
  select(popularity, grammer)
```

```
# A tibble: 8 x 2
  popularity grammer
       <dbl> <chr>
1          1 Python
2          2 C
3          3 Java
4          4 GO
5          5 <NA>
6          6 SQL
7          7 PHP
8         10 Python
```

注：可配合 `everything()` 放置“其余列”，更强大的调整列位置的函数是 `relocate()`.

1.15 题目 15（筛选行）：提取 popularity 列最大值所在的行

难度：★★

代码及运行结果：

```
df |>
  slice_max(popularity, n = 1)
```

```
# A tibble: 1 x 2
  grammer popularity
  <chr>        <dbl>
1 Python          10
```

1.16 题目 16（查看数据）：查看最后几行数据

难度：⋆

代码及运行结果：

```
df |>
  slice_tail(n = 6)   # 或者 tail(df, 6)
```

```
# A tibble: 6 x 2
  grammer popularity
  <chr>        <dbl>
1 Java             3
2 GO               4
3 <NA>             5
4 SQL              6
5 PHP              7
6 Python          10
```

注：此外，dplyr 包还提供了 slice_head() 查看前 n 行或前某比例的行，slice_sample() 随机查看 n 行或某比例的行。

1.17 题目 17（修改数据）：删除最后一行数据

难度：⋆⋆

代码及运行结果：

```
df |>
  slice(-n())
```

```
# A tibble: 7 x 2
  grammer popularity
  <chr>        <dbl>
1 Python           1
```

```
2 C                2
3 Java             3
4 GO               4
5 <NA>             5
6 SQL              6
7 PHP              7
```

1.18 题目 18（修改数据）：添加一行数据："Perl"，6

难度：★★

代码及运行结果：

```
df |>
  add_row(grammer = "Perl", popularity = 6)
```

```
# A tibble: 9 x 2
  grammer popularity
  <chr>        <dbl>
1 Python           1
2 C                2
3 Java             3
4 GO               4
5 <NA>             5
6 SQL              6
7 PHP              7
8 Python          10
9 Perl             6
```

```
# 或者
# df |>
#   bind_rows(tibble(grammer = "Perl", popularity = 6))
```

1.19 题目 19（数据整理）：对数据按 popularity 列值从到大到小排序

难度：★★

代码及运行结果：

```
df |>
  arrange(-popularity)  # 默认从小到大排序
```

```
# A tibble: 8 x 2
  grammer popularity
  <chr>        <dbl>
1 Python          10
2 PHP              7
3 SQL              6
4 <NA>             5
5 GO               4
6 Java             3
7 C                2
8 Python           1
```

1.20 题目 20（字符统计）：统计 grammer 列每个字符串的长度

难度：★★

代码及运行结果：

```
df |>
  mutate(strlen = str_length(grammer))
```

```
# A tibble: 8 x 3
  grammer popularity strlen
  <chr>        <dbl>  <int>
1 Python           1      6
2 C                2      1
```

```
3 Java              3      4
4 GO                4      2
5 <NA>              5     NA
6 SQL               6      3
7 PHP               7      3
8 Python           10      6
```

2 Part II 基础

2.1 题目 21（读取数据）：读取本地 Excel 数据

难度：⋆

代码及运行结果：

```
library(readxl)
df = read_xlsx("data/21-50 数据.xlsx")
df
```

```
# A tibble: 135 x 3
  createTime          education salary
  <dttm>              <chr>     <chr>
1 2020-03-16 11:30:18 本科      20k-35k
2 2020-03-16 10:58:48 本科      20k-40k
3 2020-03-16 10:46:39 不限      20k-35k
4 2020-03-16 10:45:44 本科      13k-20k
5 2020-03-16 10:20:41 本科      10k-20k
6 2020-03-16 10:33:48 本科      10k-18k
# i 129 more rows
```

2.2 题目 22（查看数据）：查看 df 数据的前几行

难度：⋆

代码及运行结果：

```r
head(df, 5)
```

```
# A tibble: 5 x 3
  createTime          education salary
  <dttm>              <chr>     <chr>
1 2020-03-16 11:30:18 本科      20k-35k
2 2020-03-16 10:58:48 本科      20k-40k
3 2020-03-16 10:46:39 不限      20k-35k
4 2020-03-16 10:45:44 本科      13k-20k
5 2020-03-16 10:20:41 本科      10k-20k
```

2.3 题目 23（数据计算）：将 salary 列数据转换为最大值与最小值的平均值

难度：★★★★

代码及运行结果：

```r
df = df |>
  separate(salary, into = c("low", "high"), sep = "-") |>
  mutate(salary = (parse_number(low) + parse_number(high)) * 1000 / 2) |>
  select(-c(low, high))
df
```

```
# A tibble: 135 x 3
  createTime          education salary
  <dttm>              <chr>      <dbl>
1 2020-03-16 11:30:18 本科       27500
2 2020-03-16 10:58:48 本科       30000
3 2020-03-16 10:46:39 不限       27500
4 2020-03-16 10:45:44 本科       16500
5 2020-03-16 10:20:41 本科       15000
6 2020-03-16 10:33:48 本科       14000
# i 129 more rows
```

或者来个高级的，用正则表达式提取数字，定义做计算的函数，再 purrr::map_dbl 做循环计算：

```
calc = function(x) sum(as.numeric(unlist(x))) * 1000 / 2

df |>
  mutate(salary = map_dbl(str_extract_all(salary, "\\d+"), calc))   # 结果同上（略）
```

2.4 题目 24（分组汇总）：根据学历分组，并计算平均薪资

难度：★★★

代码及运行结果：

```
df |>
  summarise(salary_avg = mean(salary), .by = education)
```

```
# A tibble: 4 x 2
  education salary_avg
  <chr>          <dbl>
1 本科           19361.
2 不限           19600
3 硕士           20643.
4 大专           10000
```

2.5 题目 25（时间转换）：将 createTime 列转换为”月-日”

难度：★★★

代码及运行结果：

```
df |>
  mutate(createTime = str_sub(createTime, 6, 10))
```

```
# A tibble: 135 x 3
  createTime education salary
  <chr>      <chr>      <dbl>
1 03-16      本科       27500
2 03-16      本科       30000
3 03-16      不限       27500
4 03-16      本科       16500
5 03-16      本科       15000
6 03-16      本科       14000
# i 129 more rows
```

2.6 题目 26（查看数据）：查看数据结构信息

难度：⋆

代码及运行结果：

```
glimpse(df)       # 或者用 str()
```

```
Rows: 135
Columns: 3
$ createTime <dttm> 2020-03-16 11:30:18, 2020-03-16 10:58:48, 2020-03-16 10:46~
$ education  <chr> "本科", "本科", "不限", "本科", "本科", "本科", "硕士", "本
科", "不限", "本科",~
$ salary     <dbl> 27500, 30000, 27500, 16500, 15000, 14000, 23000, 12500, 700~
```

```
object.size(df)   # 查看对象占用内存
```

```
5112 bytes
```

2.7 题目 27（查看数据）：查看数据汇总信息

难度：⋆

代码及运行结果：

```r
summary(df)
```

```
   createTime                     education       salary
 Min.   :2020-03-13 18:01:31   Length   :135   Min.   : 3500
 1st Qu.:2020-03-16 10:41:19   N.unique :  4   1st Qu.:14000
 Median :2020-03-16 11:00:27   N.blank  :  0   Median :17500
 Mean   :2020-03-16 10:16:35   Min.nchar:  2   Mean   :19159
 3rd Qu.:2020-03-16 11:19:03   Max.nchar:  2   3rd Qu.:25000
 Max.   :2020-03-16 11:36:07                   Max.   :45000
```

2.8 题目 28（修改列）：新增一列将 salary 离散化为三水平值

难度：★★★★

代码及运行结果：

```r
df = df |>
  mutate(class = case_when(
    salary >= 0 & salary < 5000     ~ "低",
    salary >= 5000 & salary < 20000 ~ "中",
    .default =  "高"))
df
```

```
# A tibble: 135 x 4
  createTime          education salary class
  <dttm>              <chr>      <dbl> <chr>
1 2020-03-16 11:30:18 本科       27500 高
2 2020-03-16 10:58:48 本科       30000 高
3 2020-03-16 10:46:39 不限       27500 高
4 2020-03-16 10:45:44 本科       16500 中
5 2020-03-16 10:20:41 本科       15000 中
6 2020-03-16 10:33:48 本科       14000 中
# i 129 more rows
```

- 或者用 cut() 函数：

```
df |>
  mutate(class = cut(salary,
                     breaks = c(0,5000,20000,Inf),
                     labels = c("低", "中", "高"),
                     right = FALSE))
```

- 或者用 sjmisc 包中的 rec()，和 SPSS 的重新编码一样强大。

```
library(sjmisc)
df |>
  mutate(class = rec(salary,
    rec = "min:5000 = 低; 5000:20000 = 中; 20000:max = 高"))
```

2.9 题目 29（数据整理）：按 salary 列对数据降序排列

难度：★★

代码及运行结果：

```
df |>
  arrange(-salary)
```

```
# A tibble: 135 x 4
  createTime          education salary class
  <dttm>              <chr>      <dbl> <chr>
1 2020-03-16 11:30:17 本科       45000 高
2 2020-03-16 11:04:00 本科       40000 高
3 2020-03-16 10:36:57 本科       37500 高
4 2020-03-16 11:01:39 本科       37500 高
5 2020-03-16 09:54:47 硕士       37500 高
6 2020-03-16 11:01:22 本科       35000 高
# i 129 more rows
```

2.10 题目 30（筛选行）：提取第 33 行数据

难度：⋆

代码及运行结果：

```
df |>
  slice(33)       # 或者 df[33,]
```

```
# A tibble: 1 x 4
  createTime          education salary class
  <dttm>              <chr>      <dbl> <chr>
1 2020-03-16 10:07:25 硕士       22500 高
```

2.11 题目 31（数据计算）：计算 salary 列的中位数

难度：⋆

代码及运行结果：

```
df |>
  summarise(med = median(salary))  # 或者 median(df$salary)
```

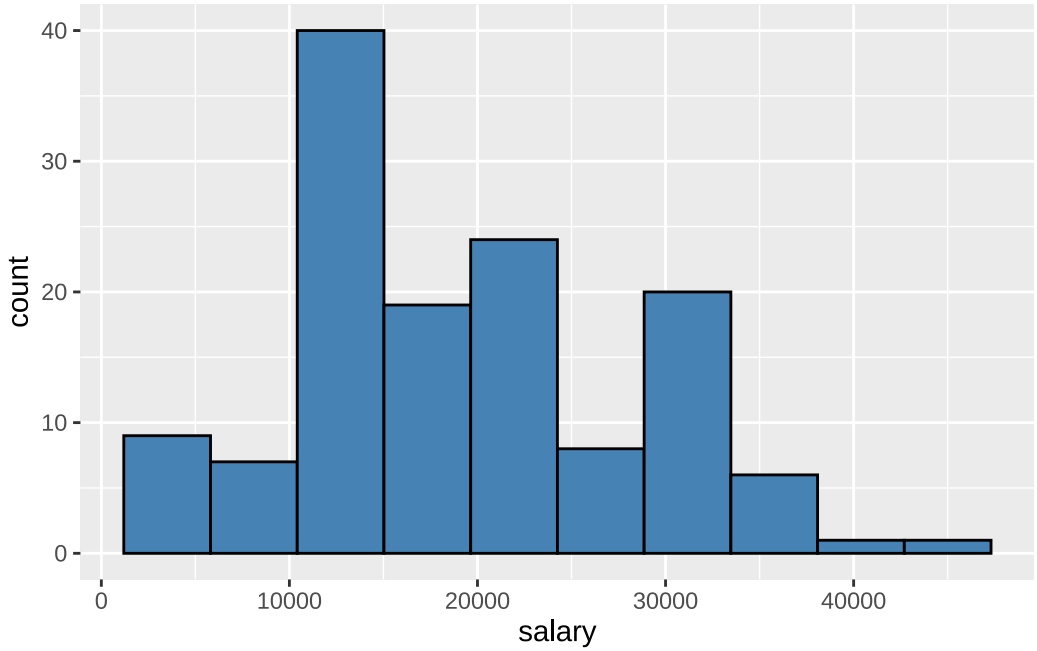
```
# A tibble: 1 x 1
    med
  <dbl>
1 17500
```

2.12 题目 32（数据可视化）：绘制 salary 的频率分布直方图

难度：⋆⋆⋆

代码及运行结果：

```r
df |>
  ggplot(aes(salary)) +
  geom_histogram(bins = 10, fill = "steelblue", color = "black")
```

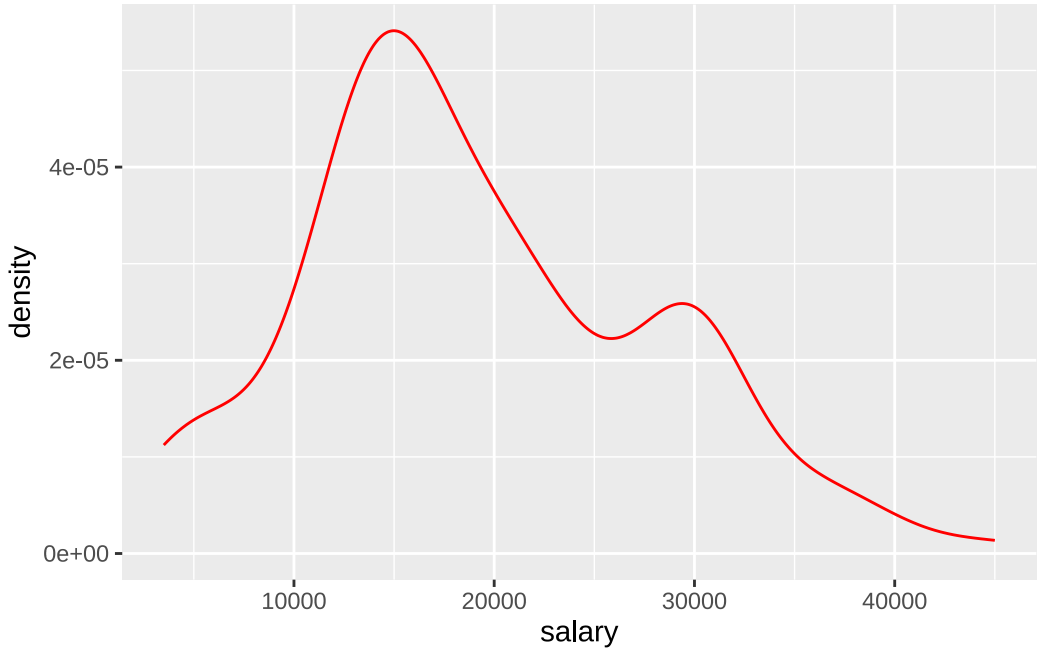


2.13 题目 33 (数据可视化)：绘制 salary 的频率密度曲线图

难度：★★★

代码及运行结果：

```r
df |>
  ggplot(aes(salary)) +
  geom_density(color = "red")
```



2.14 题目 34（数据删除）：删除最后一列 class

难度：★

代码及运行结果：

```
df |>
  select(-class)
```

```
# A tibble: 135 x 3
  createTime          education salary
  <dttm>              <chr>      <dbl>
1 2020-03-16 11:30:18 本科       27500
2 2020-03-16 10:58:48 本科       30000
3 2020-03-16 10:46:39 不限       27500
4 2020-03-16 10:45:44 本科       16500
5 2020-03-16 10:20:41 本科       15000
6 2020-03-16 10:33:48 本科       14000
# i 129 more rows
```

```
# 或者
# df |>
#  select(-last_col())  # 同 last_col(0)
```

2.15 题目 35（数据操作）：将 df 的第 1 列与第 2 列合并为新的一列

难度：★★

代码及运行结果：

```
df |>
  unite("newcol", 1:2, sep = " ")
```

```
# A tibble: 135 x 3
  newcol                   salary class
  <chr>                     <dbl> <chr>
1 2020-03-16 11:30:18 本科  27500 高
2 2020-03-16 10:58:48 本科  30000 高
3 2020-03-16 10:46:39 不限  27500 高
4 2020-03-16 10:45:44 本科  16500 中
5 2020-03-16 10:20:41 本科  15000 中
6 2020-03-16 10:33:48 本科  14000 中
# i 129 more rows
```

2.16 题目 36（数据操作）：将 education 列与 salary 列合并为新的一列

难度：★★

代码及运行结果：

```
df |>
  unite("newcol", c(education, salary), sep = " ")
```

```
# A tibble: 135 x 3
  createTime          newcol     class
  <dttm>              <chr>      <chr>
1 2020-03-16 11:30:18 本科 27500 高
2 2020-03-16 10:58:48 本科 30000 高
3 2020-03-16 10:46:39 不限 27500 高
4 2020-03-16 10:45:44 本科 16500 中
5 2020-03-16 10:20:41 本科 15000 中
6 2020-03-16 10:33:48 本科 14000 中
# i 129 more rows
```

2.17 题目 37（数据计算）：计算 salary 最大值与最小值之差

难度：★★

代码及运行结果：

```
df |>
  summarise(range = max(salary) - min(salary))
```

```
# A tibble: 1 x 1
  range
  <dbl>
1 41500
```

```
# 或者 max(df$salary) - min(df$salary)
```

2.18 题目 38（数据操作）：将第一行与最后一行拼接

难度：★★

代码及运行结果：

```r
df |>
  slice(1, n())
```

```
# A tibble: 2 x 4
  createTime          education salary class
  <dttm>              <chr>      <dbl> <chr>
1 2020-03-16 11:30:18 本科       27500 高
2 2020-03-16 11:19:38 本科       30000 高
```

2.19 题目 39（数据操作）：将第 8 行添加到末尾

难度：★★

代码及运行结果：

```r
df |>
  slice(c(1:n(), 8))
```

```
# A tibble: 136 x 4
  createTime          education salary class
  <dttm>              <chr>      <dbl> <chr>
1 2020-03-16 11:30:18 本科       27500 高
2 2020-03-16 10:58:48 本科       30000 高
3 2020-03-16 10:46:39 不限       27500 高
4 2020-03-16 10:45:44 本科       16500 中
5 2020-03-16 10:20:41 本科       15000 中
6 2020-03-16 10:33:48 本科       14000 中
# i 130 more rows
```

```r
# 或者
# df |> add_row(df[8, ])
```

2.20 题目 40（查看数据）：查看每一列的数据类型

难度：⋆

代码及运行结果：

```
glimpse(df)     # 或者用 str()
```

```
Rows: 135
Columns: 4
$ createTime <dttm> 2020-03-16 11:30:18, 2020-03-16 10:58:48, 2020-03-16 10:46~
$ education  <chr> "本科", "本科", "不限", "本科", "本科", "本科", "硕士", "本
科", "不限", "本科",~
$ salary     <dbl> 27500, 30000, 27500, 16500, 15000, 14000, 23000, 12500, 700~
$ class      <chr> "高", "高", "高", "中", "中", "中", "高", "中", "中", "中", "
高", "中",~
```

2.21 题目 41（数据操作）：将 createTime 列设置为行索引

难度：⋆⋆

代码及运行结果：

```
df |>
  distinct(createTime, .keep_all = TRUE) |>
  column_to_rownames("createTime")
```

```
                    education salary class
2020-03-16 11:30:18      本科  27500     高
2020-03-16 10:58:48      本科  30000     高
2020-03-16 10:46:39      不限  27500     高
2020-03-16 10:45:44      本科  16500     中
2020-03-16 10:20:41      本科  15000     中
2020-03-16 10:33:48      本科  14000     中
2020-03-16 10:11:54      硕士  23000     高
```

2020-03-16 09:49:12	本科	12500	中
2020-03-16 09:25:48	不限	7000	中
2020-03-16 09:35:50	本科	16000	中
2020-03-16 10:34:19	本科	20000	高
2020-03-16 09:30:40	本科	10000	中
2020-03-16 11:30:17	本科	30000	高
2020-03-16 10:54:56	本科	25000	高
2020-03-15 12:14:45	本科	30000	高
2020-03-16 10:52:14	硕士	12500	中
2020-03-16 10:36:57	本科	37500	高
2020-03-16 11:01:23	本科	12500	中
2020-03-16 11:01:22	本科	35000	高
2020-03-16 11:03:56	本科	20000	高
2020-03-16 11:00:27	本科	11500	中
2020-03-16 11:04:44	本科	30000	高
2020-03-16 11:30:15	本科	20000	高
2020-03-16 11:02:12	本科	22500	高
2020-03-16 11:19:06	本科	15000	中
2020-03-16 11:04:45	本科	30000	高
2020-03-16 11:04:49	本科	30000	高
2020-03-16 10:27:11	本科	17500	中
2020-03-16 10:43:58	硕士	22500	高
2020-03-16 11:01:46	本科	22500	高
2020-03-16 10:07:25	硕士	22500	高
2020-03-16 11:04:00	本科	30000	高
2020-03-16 10:43:53	本科	24000	高
2020-03-13 18:01:31	本科	16000	中
2020-03-16 10:15:50	本科	22500	高
2020-03-16 11:01:58	不限	30000	高
2020-03-16 11:35:36	本科	30000	高
2020-03-16 11:30:14	本科	20000	高
2020-03-16 11:30:02	本科	30000	高
2020-03-16 11:18:00	本科	12500	中
2020-03-16 11:16:49	本科	30000	高
2020-03-16 10:59:08	本科	12000	中

2020-03-16 11:35:39	本科	11500	中
2020-03-16 11:30:09	本科	17500	中
2020-03-16 11:29:50	本科	16000	中
2020-03-16 11:35:34	本科	12500	中
2020-03-16 11:28:48	本科	13500	中
2020-03-16 11:32:46	本科	15000	中
2020-03-16 11:33:00	本科	16500	中
2020-03-16 11:09:18	本科	15000	中
2020-03-16 11:35:33	本科	25000	高
2020-03-16 11:01:07	本科	22500	高
2020-03-16 10:57:55	本科	20000	高
2020-03-16 10:51:27	本科	9500	中
2020-03-16 10:46:50	本科	22500	高
2020-03-16 10:54:10	本科	17500	中
2020-03-16 10:41:20	本科	35000	高
2020-03-16 10:41:19	本科	25000	高
2020-03-16 10:46:31	本科	18000	中
2020-03-16 10:34:43	本科	27500	高
2020-03-16 10:43:47	本科	14000	中
2020-03-16 10:34:27	本科	17500	中
2020-03-16 10:26:23	本科	15000	中
2020-03-16 10:18:39	本科	12000	中
2020-03-16 11:20:44	本科	6500	中
2020-03-16 11:19:03	本科	14000	中
2020-03-16 11:17:58	本科	14000	中
2020-03-16 11:10:42	本科	20000	高
2020-03-16 10:18:35	本科	25000	高
2020-03-16 11:01:08	本科	15000	中
2020-03-16 11:30:10	本科	32500	高
2020-03-16 10:52:45	本科	16000	中
2020-03-16 10:50:23	本科	5000	中
2020-03-16 10:43:49	本科	30000	高
2020-03-16 10:43:46	本科	14000	中
2020-03-16 10:26:50	本科	15000	中
2020-03-16 11:33:08	本科	15000	中

2020-03-16 10:27:10	硕士	14000	中
2020-03-16 10:44:41	本科	30000	高
2020-03-16 11:12:04	本科	15000	中
2020-03-16 10:44:23	不限	3500	低
2020-03-16 10:27:45	本科	30000	高
2020-03-16 11:01:39	本科	37500	高
2020-03-16 10:21:52	大专	15000	中
2020-03-16 11:21:24	本科	16000	中
2020-03-16 10:26:12	大专	5000	中
2020-03-16 10:59:15	本科	16000	中
2020-03-16 11:13:50	本科	12500	中
2020-03-16 11:13:16	本科	26500	高
2020-03-16 10:48:43	本科	3500	低
2020-03-16 11:32:07	本科	8500	中
2020-03-16 11:19:36	本科	30000	高
2020-03-16 11:19:38	本科	30000	高
2020-03-16 09:28:37	本科	18500	中
2020-03-16 10:09:18	本科	11500	中
2020-03-16 11:33:13	本科	5000	中
2020-03-16 10:00:03	本科	20000	高
2020-03-16 09:44:05	硕士	12500	中
2020-03-16 10:57:27	本科	22500	高
2020-03-16 09:46:26	本科	20000	高
2020-03-16 11:36:07	本科	14000	中
2020-03-16 09:54:47	硕士	37500	高
2020-03-16 10:48:32	本科	30000	高

注：行索引不允许有重复，所以先做了一步去重。

2.22 题目 42 (数据创建)：生成一个和 df 长度相同的随机数数据框

难度：⋆⋆

代码及运行结果：

```r
set.seed(123)
df1 = tibble(rnums = sample(10, nrow(df), replace = TRUE))
df1
```

```
# A tibble: 135 x 1
  rnums
  <int>
1     3
2     3
3    10
4     2
5     6
6     5
# i 129 more rows
```

2.23 题目 43（数据连接）：将上面生成的数据框与 df 按列合并

难度：★★

代码及运行结果：

```r
df = bind_cols(df, df1)
df
```

```
# A tibble: 135 x 5
  createTime          education salary class rnums
  <dttm>              <chr>      <dbl> <chr> <int>
1 2020-03-16 11:30:18 本科       27500 高        3
2 2020-03-16 10:58:48 本科       30000 高        3
3 2020-03-16 10:46:39 不限       27500 高       10
4 2020-03-16 10:45:44 本科       16500 中        2
5 2020-03-16 10:20:41 本科       15000 中        6
6 2020-03-16 10:33:48 本科       14000 中        5
# i 129 more rows
```

注：实际上，42，43 题应该合并成一个题，这是数据操作中最常规的修改列：

```
df |>
  mutate(rnums = sample(10, n(), replace = TRUE))
```

```
# A tibble: 135 x 5
  createTime          education salary class rnums
  <dttm>              <chr>      <dbl> <chr> <int>
1 2020-03-16 11:30:18 本科       27500 高       10
2 2020-03-16 10:58:48 本科       30000 高        4
3 2020-03-16 10:46:39 不限       27500 高        9
4 2020-03-16 10:45:44 本科       16500 中        7
5 2020-03-16 10:20:41 本科       15000 中        8
6 2020-03-16 10:33:48 本科       14000 中        5
# i 129 more rows
```

2.24 题目 44（修改列）：生成新列 new 为 salary 列减去随机数列

难度：★★

代码及运行结果：

```
df = df |>
  mutate(new = salary - rnums)
df
```

```
# A tibble: 135 x 6
  createTime          education salary class rnums   new
  <dttm>              <chr>      <dbl> <chr> <int> <dbl>
1 2020-03-16 11:30:18 本科       27500 高        3 27497
2 2020-03-16 10:58:48 本科       30000 高        3 29997
3 2020-03-16 10:46:39 不限       27500 高       10 27490
4 2020-03-16 10:45:44 本科       16500 中        2 16498
5 2020-03-16 10:20:41 本科       15000 中        6 14994
6 2020-03-16 10:33:48 本科       14000 中        5 13995
# i 129 more rows
```

2.25 题目 45（检查缺失值）：检查数据中是否含有任何缺失值

难度：★★

代码及运行结果：

```
anyNA(df)
```

```
[1] FALSE
```

```
anyNA(df$salary)
```

```
[1] FALSE
```

注：naniar 包提供了更强大的探索缺失值及缺失模式的函数，其中 miss_var_summary() 和 miss_case_summary() 可检查各列和各行缺失情况。

2.26 题目 46（类型转换）：将 salary 列的类型转换为浮点数

难度：★★

代码及运行结果：

```
df |>
  mutate(rnums = as.double(rnums))
```

```
# A tibble: 135 x 6
  createTime          education salary class rnums   new
  <dttm>              <chr>      <dbl> <chr> <dbl> <dbl>
1 2020-03-16 11:30:18 本科       27500 高        3 27497
2 2020-03-16 10:58:48 本科       30000 高        3 29997
3 2020-03-16 10:46:39 不限       27500 高       10 27490
4 2020-03-16 10:45:44 本科       16500 中        2 16498
5 2020-03-16 10:20:41 本科       15000 中        6 14994
6 2020-03-16 10:33:48 本科       14000 中        5 13995
# i 129 more rows
```

2.27 题目 47（数据汇总）：计算 salary 列大于 10000 的次数

难度：★★★

代码及运行结果：

```
df |>
  count(salary > 10000)
```

```
# A tibble: 2 x 2
  `salary > 10000`     n
  <lgl>            <int>
1 FALSE               16
2 TRUE               119
```

或者用

```
df |>
  summarise(n = sum(salary > 10000))
```

```
# A tibble: 1 x 1
      n
  <int>
1   119
```

2.28 题目 48（统计频数）：查看每种学历出现的次数

难度：★★

代码及运行结果：

```
df |>
  count(education)
```

```
# A tibble: 4 x 2
  education     n
  <chr>     <int>
1 不限          5
2 大专          4
3 本科        119
4 硕士          7
```

2.29 题目 49（数据汇总）：查看 education 列共有几种学历

难度：★★

代码及运行结果：

```
df |>
  distinct(education)
```

```
# A tibble: 4 x 1
  education
  <chr>
1 本科
2 不限
3 硕士
4 大专
```

2.30 题目 50（筛选行）：提取 salary 与 new 列之和大于 60000 的最后 3 行

难度：★★★★

代码及运行结果：

```
df |>
  filter(salary + new > 60000) |>
  slice_tail(n = 3)
```

```
# A tibble: 3 x 6
  createTime          education salary class rnums   new
  <dttm>              <chr>      <dbl> <chr> <int> <dbl>
1 2020-03-16 10:41:20 本科       35000 高        8 34992
2 2020-03-16 11:01:39 本科       37500 高        7 37493
3 2020-03-16 09:54:47 硕士       37500 高        4 37496
```

3 Part III 提高

3.1 题目 51（读取数据）：使用绝对路径读取本地 Excel 数据

难度：⋆

代码及运行结果：

```
library(readxl)
df = read_xls("data/51-80 数据.xls")
df
```

```
# A tibble: 327 x 18
  代码       简称     日期                `前收盘价(元)` `开盘价(元)` `最高价(元)`
  <chr>      <chr>    <dttm>                       <dbl>        <dbl>        <dbl>
1 600000.SH 浦发银行~~ 2016-01-04 00:00:00          16.1         16.1         16.1
2 600000.SH 浦发银行~~ 2016-01-05 00:00:00          15.7         15.5         16.0
3 600000.SH 浦发银行~~ 2016-01-06 00:00:00          15.9         15.8         16.0
4 600000.SH 浦发银行~~ 2016-01-07 00:00:00          16.0         15.7         15.8
5 600000.SH 浦发银行~~ 2016-01-08 00:00:00          15.5         15.7         15.8
6 600000.SH 浦发银行~~ 2016-01-11 00:00:00          15.4         15.2         15.4
# i 321 more rows
# i 12 more variables: `最低价(元)` <dbl>, `收盘价(元)` <dbl>,
#   `成交量(股)` <chr>, `成交金额(元)` <chr>, `涨跌(元)` <dbl>,
#   `涨跌幅(%)` <dbl>, `均价(元)` <chr>, `换手率(%)` <chr>,
#   `A股流通市值(元)` <dbl>, `总市值(元)` <dbl>, `A股流通股本(股)` <dbl>,
#   市盈率 <dbl>
```

3.2 题目 52（查看数据）：查看数据框的前 3 行

难度：⋆

代码及运行结果：

```
head(df, 3)
```

```
# A tibble: 3 x 18
  代码       简称     日期                `前收盘价(元)` `开盘价(元)` `最高价(元)`
  <chr>      <chr>    <dttm>                       <dbl>         <dbl>         <dbl>
1 600000.SH 浦发银行~~ 2016-01-04 00:00:00           16.1          16.1          16.1
2 600000.SH 浦发银行~~ 2016-01-05 00:00:00           15.7          15.5          16.0
3 600000.SH 浦发银行~~ 2016-01-06 00:00:00           15.9          15.8          16.0
# i 12 more variables: `最低价(元)` <dbl>, `收盘价(元)` <dbl>,
#   `成交量(股)` <chr>, `成交金额(元)` <chr>, `涨跌(元)` <dbl>,
#   `涨跌幅(%)` <dbl>, `均价(元)` <chr>, `换手率(%)` <chr>,
#   `A股流通市值(元)` <dbl>, `总市值(元)` <dbl>, `A股流通股本(股)` <dbl>,
#   市盈率 <dbl>
```

说明：当前数据不包含缺失值，接下来关于缺失值的题目 53-56，改用自带的 starwars 数据演示。

3.3 题目 53（查看缺失值）：查看每列数据缺失值情况

难度：⋆⋆

代码及运行结果：

```
map_int(starwars, \(x) sum(is.na(x)))
```

```
      name     height       mass hair_color skin_color  eye_color birth_year
         0          6         28          5          0          0         44
       sex     gender  homeworld    species      films   vehicles  starships
         4          4         10          4          0          0          0
```

注：也可以用 naniar 包中的 miss_var_summary() 函数。

3.4 题目 54（查看缺失值）：查看日期列含有缺失值的行

难度：★★

代码及运行结果：

```
starwars |>
  filter(is.na(hair_color))
```

```
# A tibble: 5 x 14
  name      height  mass hair_color skin_color eye_color birth_year sex   gender
  <chr>      <int> <dbl> <chr>      <chr>      <chr>           <dbl> <chr> <chr>
1 C-3PO        167    75 <NA>       gold       yellow            112 none  mascu~
2 R2-D2         96    32 <NA>       white, bl~ red                33 none  mascu~
3 R5-D4         97    32 <NA>       white, red red                NA none  mascu~
4 Greedo       173    74 <NA>       green      black              44 male  mascu~
5 Jabba De~    175  1358 <NA>       green-tan~ orange            600 herm~ mascu~
# i 5 more variables: homeworld <chr>, species <chr>, films <list>,
#   vehicles <list>, starships <list>
```

3.5 题目 55（查看缺失值）：查看每列缺失值在哪些行

难度：★★★

代码及运行结果：

```
map(starwars, \(x) which(is.na(x)))
```

```
$name
integer(0)

$height
[1] 28 83 84 85 86 87

$mass
```

```
 [1] 12 27 28 33 37 38 39 41 42 44 48 53 55 56 58 60 61 65 67 72 73 74 76 83 84
[26] 85 86 87

$hair_color
[1]  2  3  8 15 16

$skin_color
integer(0)

$eye_color
integer(0)

$birth_year
 [1]  8 18 28 30 32 36 37 38 39 40 44 46 47 48 49 52 53 54 55 56 58 59 60 62 65
[26] 69 70 71 72 73 74 75 76 77 78 79 80 81 82 83 84 85 86 87

$sex
[1] 18 59 60 81

$gender
[1] 18 59 60 81

$homeworld
 [1] 19 22 28 31 74 83 84 85 86 87

$species
[1] 18 59 60 81

$films
integer(0)

$vehicles
integer(0)

$starships
```

```
integer(0)
```

3.6 题目 56（缺失值处理）：删除所有存在缺失值的行

难度：★★

代码及运行结果：

```
starwars |>
  drop_na()
```

```
# A tibble: 29 x 14
  name      height  mass hair_color skin_color eye_color birth_year sex   gender
  <chr>      <int> <dbl> <chr>      <chr>      <chr>          <dbl> <chr> <chr>
1 Luke Sky~    172    77 blond      fair       blue            19   male  mascu~
2 Darth Va~    202   136 none       white      yellow          41.9 male  mascu~
3 Leia Org~    150    49 brown      light      brown           19   fema~ femin~
4 Owen Lars    178   120 brown, gr~ light      blue            52   male  mascu~
5 Beru Whi~    165    75 brown      light      blue            47   fema~ femin~
6 Biggs Da~    183    84 black      light      brown           24   male  mascu~
# i 23 more rows
# i 5 more variables: homeworld <chr>, species <chr>, films <list>,
#   vehicles <list>, starships <list>
```

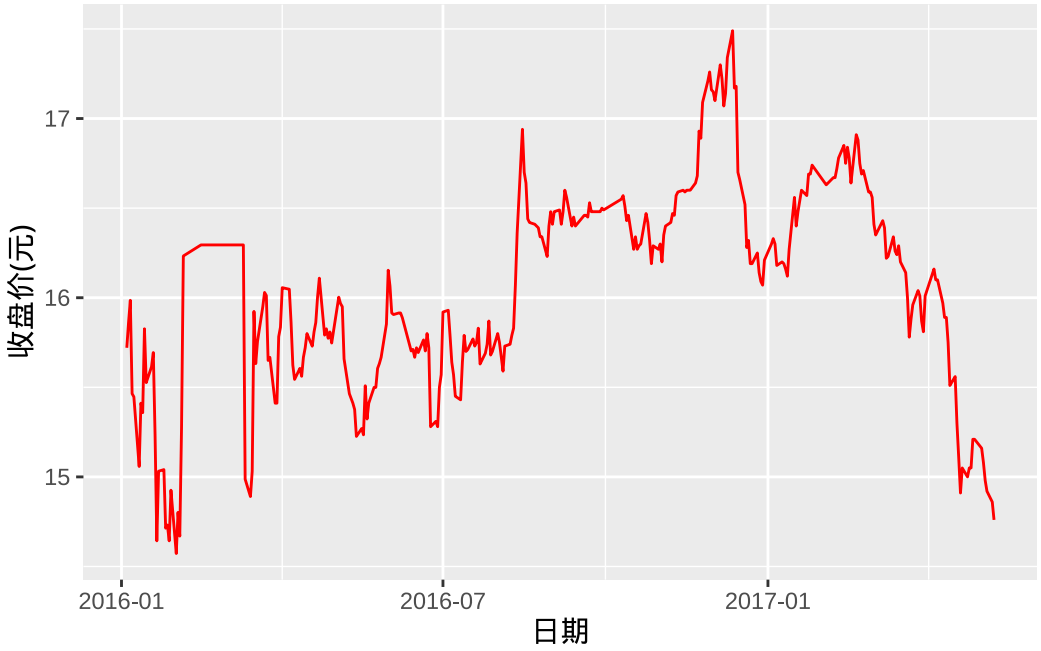
注：若要删除某些列包含缺失值的行，提供列名即可。

3.7 题目 57（数据可视化）：绘制收盘价的折线图

难度：★★

代码及运行结果：

```
df |>
    ggplot(aes(日期, `收盘价（元）`)) +
    geom_line(color = "red")
```

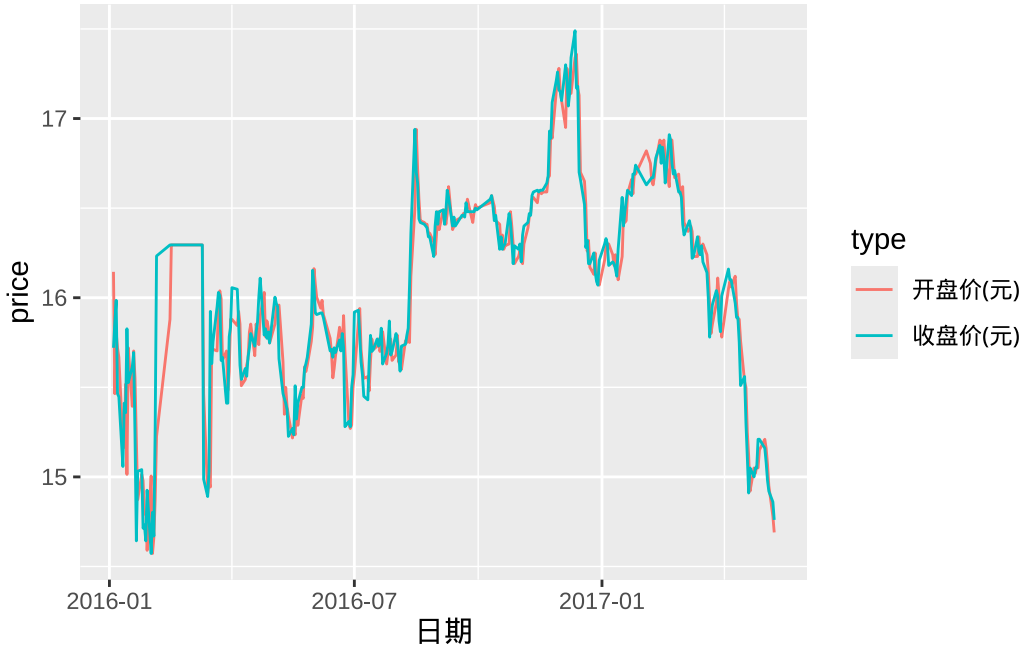


3.8 题目 58（数据可视化）：同时绘制开盘价与收盘价

难度：★★★

代码及运行结果：

```
df |>
  select(日期, `开盘价（元）`, `收盘价（元）`) |>
  pivot_longer(-日期, names_to = "type", values_to = "price") |>
  ggplot(aes(日期, price, color = type)) +
  geom_line()
```



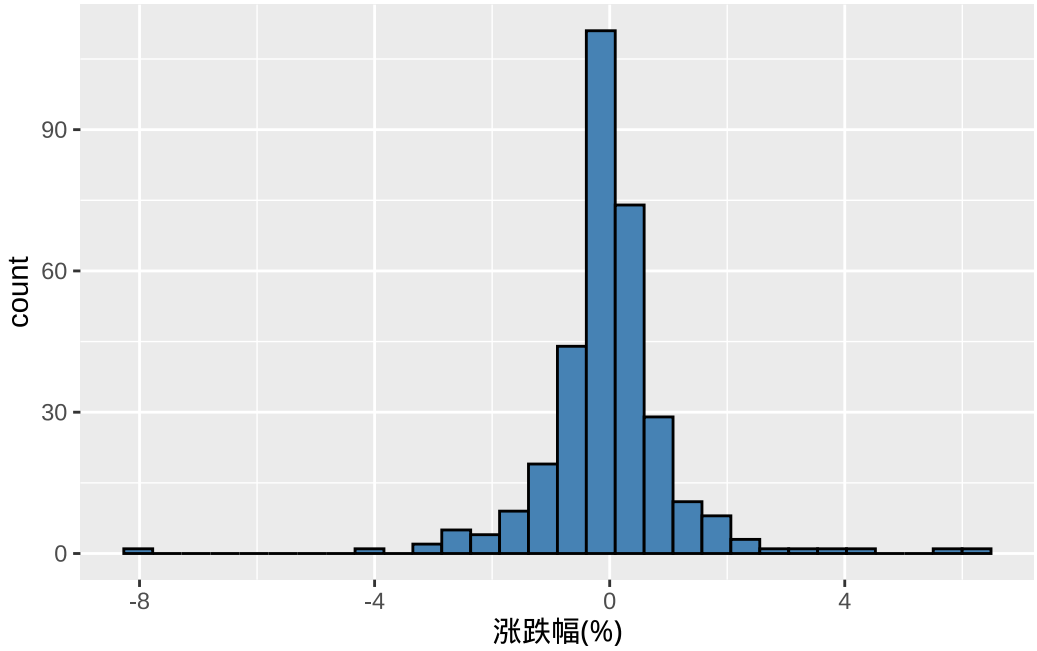
注：为了自动添加图例，先对数据做了宽变长转换。

3.9 题目 59（数据可视化）：绘制涨跌幅的直方图

难度：★★

代码及运行结果：

```
df |>
  ggplot(aes(`涨跌幅(%)`)) +
  geom_histogram(fill = "steelblue", color = "black")
```

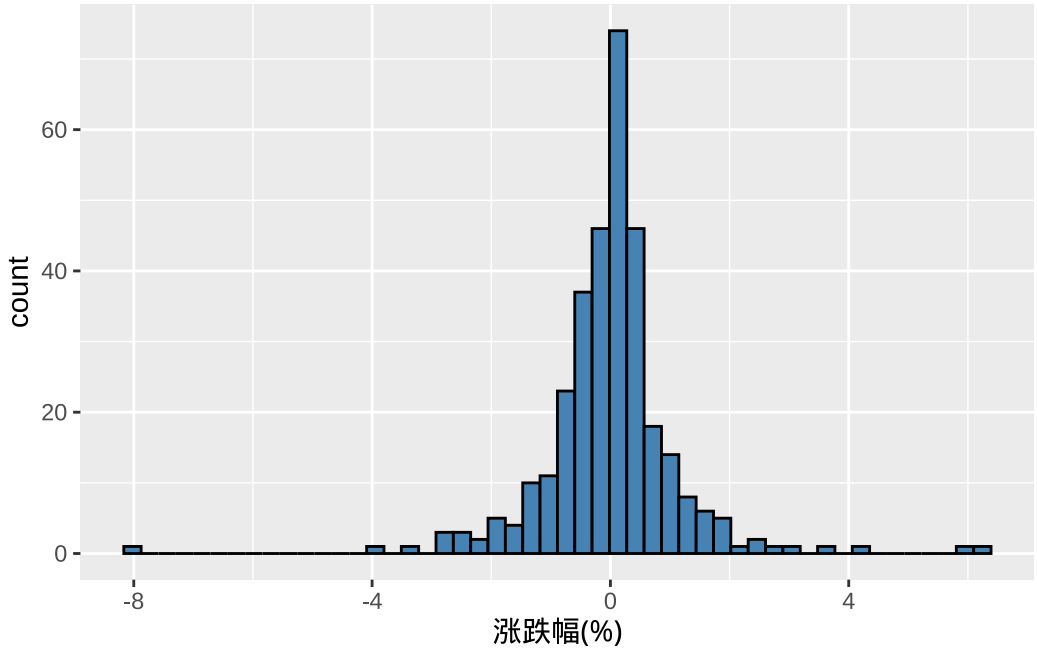


3.10 题目 60（数据可视化）：让直方图更细致

难度：★★

代码及运行结果：

```
df |>
  ggplot(aes(`涨跌幅(%)`)) +
  geom_histogram(bins = 50, fill = "steelblue", color = "black")
```



3.11 题目 61（数据创建）：用 df 的列名创建数据框

难度：★★

代码及运行结果：

```
tibble(Name = names(df))
```

```
# A tibble: 18 x 1
  Name
  <chr>
1 代码
2 简称
3 日期
4 前收盘价(元)
5 开盘价(元)
6 最高价(元)
# i 12 more rows
```

3.12 题目 62（异常值处理）：输出所有换手率不是数值的行

难度：★★

代码及运行结果：

```
df |>
  mutate(`换手率(%)` = parse_number(`换手率(%)`)) |>
  filter(is.na(`换手率(%)`))
```

```
# A tibble: 18 x 18
  代码       简称     日期                `前收盘价(元)` `开盘价(元)` `最高价(元)`
  <chr>      <chr>    <dttm>                       <dbl>        <dbl>        <dbl>
1 600000.SH 浦发银行~~ 2016-02-16 00:00:00           16.3         16.3         16.3
2 600000.SH 浦发银行~~ 2016-02-17 00:00:00           16.3         16.3         16.3
3 600000.SH 浦发银行~~ 2016-02-18 00:00:00           16.3         16.3         16.3
4 600000.SH 浦发银行~~ 2016-02-19 00:00:00           16.3         16.3         16.3
5 600000.SH 浦发银行~~ 2016-02-22 00:00:00           16.3         16.3         16.3
6 600000.SH 浦发银行~~ 2016-02-23 00:00:00           16.3         16.3         16.3
# i 12 more rows
# i 12 more variables: `最低价(元)` <dbl>, `收盘价(元)` <dbl>,
#   `成交量(股)` <chr>, `成交金额(元)` <chr>, `涨跌(元)` <dbl>,
#   `涨跌幅(%)` <dbl>, `均价(元)` <chr>, `换手率(%)` <dbl>,
#   `A股流通市值(元)` <dbl>, `总市值(元)` <dbl>, `A股流通股本(股)` <dbl>,
#   市盈率 <dbl>
```

3.13 题目 63（异常值处理）：输出所有换手率为 -- 的行

难度：★★

代码及运行结果：

```
df |>
  filter(`换手率(%)` == "--")
```

```
# A tibble: 18 x 18
  代码       简称     日期                `前收盘价(元)` `开盘价(元)` `最高价(元)`
  <chr>      <chr>    <dttm>                       <dbl>        <dbl>        <dbl>
1 600000.SH 浦发银行~~ 2016-02-16 00:00:00          16.3         16.3         16.3
2 600000.SH 浦发银行~~ 2016-02-17 00:00:00          16.3         16.3         16.3
3 600000.SH 浦发银行~~ 2016-02-18 00:00:00          16.3         16.3         16.3
4 600000.SH 浦发银行~~ 2016-02-19 00:00:00          16.3         16.3         16.3
5 600000.SH 浦发银行~~ 2016-02-22 00:00:00          16.3         16.3         16.3
6 600000.SH 浦发银行~~ 2016-02-23 00:00:00          16.3         16.3         16.3
# i 12 more rows
# i 12 more variables: `最低价(元)` <dbl>, `收盘价(元)` <dbl>,
#   `成交量(股)` <chr>, `成交金额(元)` <chr>, `涨跌(元)` <dbl>,
#   `涨跌幅(%)` <dbl>, `均价(元)` <chr>, `换手率(%)` <chr>,
#   `A股流通市值(元)` <dbl>, `总市值(元)` <dbl>, `A股流通股本(股)` <dbl>,
#   市盈率 <dbl>
```

3.14 题目 64（数据操作）：重置 df 的行号

难度：⋆

代码及运行结果：

```r
rownames(df) = NULL     # R 中无行号就是数字索引
```

3.15 题目 65（异常值处理）：删除所有换手率为非数值的行

难度：⋆⋆

代码及运行结果：

```r
df |>
  mutate(`换手率 (%)` = parse_number(`换手率 (%)`)) |>
  filter(!is.na(`换手率 (%)`))
```

```
# A tibble: 309 x 18
  代码      简称    日期                `前收盘价(元)` `开盘价(元)` `最高价(元)`
  <chr>     <chr>   <dttm>                       <dbl>        <dbl>        <dbl>
1 600000.SH 浦发银行~~ 2016-01-04 00:00:00        16.1         16.1         16.1
2 600000.SH 浦发银行~~ 2016-01-05 00:00:00        15.7         15.5         16.0
3 600000.SH 浦发银行~~ 2016-01-06 00:00:00        15.9         15.8         16.0
4 600000.SH 浦发银行~~ 2016-01-07 00:00:00        16.0         15.7         15.8
5 600000.SH 浦发银行~~ 2016-01-08 00:00:00        15.5         15.7         15.8
6 600000.SH 浦发银行~~ 2016-01-11 00:00:00        15.4         15.2         15.4
# i 303 more rows
# i 12 more variables: `最低价(元)` <dbl>, `收盘价(元)` <dbl>,
#   `成交量(股)` <chr>, `成交金额(元)` <chr>, `涨跌(元)` <dbl>,
#   `涨跌幅(%)` <dbl>, `均价(元)` <chr>, `换手率(%)` <dbl>,
#   `A股流通市值(元)` <dbl>, `总市值(元)` <dbl>, `A股流通股本(股)` <dbl>,
#   市盈率 <dbl>
```

```
df = df |>
  mutate(across(4:18, as.numeric), 日期 = as_date(日期))
df
```

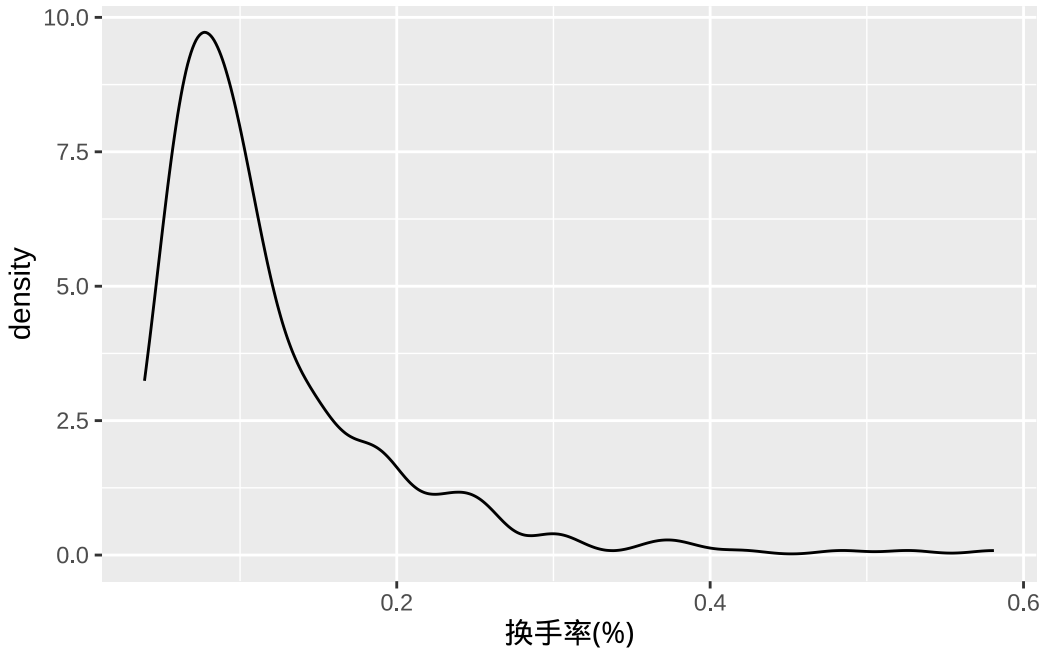
```
# A tibble: 327 x 18
  代码     简称   日期       `前收盘价(元)` `开盘价(元)` `最高价(元)` `最低价(元)`
  <chr>    <chr>  <date>              <dbl>        <dbl>        <dbl>        <dbl>
1 600000~ 浦发银行~ 2016-01-04          16.1         16.1         16.1         15.5
2 600000~ 浦发银行~ 2016-01-05          15.7         15.5         16.0         15.4
3 600000~ 浦发银行~ 2016-01-06          15.9         15.8         16.0         15.6
4 600000~ 浦发银行~ 2016-01-07          16.0         15.7         15.8         15.4
5 600000~ 浦发银行~ 2016-01-08          15.5         15.7         15.8         14.9
6 600000~ 浦发银行~ 2016-01-11          15.4         15.2         15.4         15.0
# i 321 more rows
# i 11 more variables: `收盘价(元)` <dbl>, `成交量(股)` <dbl>,
#   `成交金额(元)` <dbl>, `涨跌(元)` <dbl>, `涨跌幅(%)` <dbl>,
#   `均价(元)` <dbl>, `换手率(%)` <dbl>, `A股流通市值(元)` <dbl>,
#   `总市值(元)` <dbl>, `A股流通股本(股)` <dbl>, 市盈率 <dbl>
```

3.16 题目 66（数据可视化）：绘制换手率的密度曲线

难度：★★

代码及运行结果：

```
df |>
  ggplot(aes(`换手率 (%)`)) +
  geom_density()
```



3.17 题目 67（数据计算）：计算前一天与后一天收盘价的差值

难度：★★

代码及运行结果：

```
df |>
  mutate(delta = `收盘价 (元)` - lag(`收盘价 (元)`)) |>
  select(日期, `收盘价 (元)`, delta)
```

```
# A tibble: 327 x 3
  日期       `收盘价(元)`   delta
  <date>            <dbl>   <dbl>
1 2016-01-04         15.7 NA
2 2016-01-05         15.9  0.141
3 2016-01-06         16.0  0.124
4 2016-01-07         15.5 -0.521
5 2016-01-08         15.4 -0.0177
6 2016-01-11         15.1 -0.389
# i 321 more rows
```

3.18 题目 68 (数据计算)：计算前一天与后一天收盘价的变化率

难度：★★

代码及运行结果：

```
df |>
  mutate(change = (`收盘价(元)` - lag(`收盘价(元)`)) / `收盘价(元)`) |>
  select(日期, `收盘价(元)`, change)
```

```
# A tibble: 327 x 3
  日期       `收盘价(元)`   change
  <date>            <dbl>    <dbl>
1 2016-01-04         15.7 NA
2 2016-01-05         15.9  0.00891
3 2016-01-06         16.0  0.00774
4 2016-01-07         15.5 -0.0337
5 2016-01-08         15.4 -0.00115
6 2016-01-11         15.1 -0.0258
# i 321 more rows
```

3.19 题目 69 (数据操作)：设置日期为行索引

难度：★

代码及运行结果：

```
df |>
  column_to_rownames("日期") |>   # 将从 tibble 变成 data.frame
  head()
```

```
                代码      简称 前收盘价(元) 开盘价(元) 最高价(元) 最低价(元)
2016-01-04 600000.SH 浦发银行      16.1356    16.1444    16.1444    15.4997
2016-01-05 600000.SH 浦发银行      15.7205    15.4644    15.9501    15.3672
2016-01-06 600000.SH 浦发银行      15.8618    15.8088    16.0208    15.6234
2016-01-07 600000.SH 浦发银行      15.9855    15.7205    15.8088    15.3672
2016-01-08 600000.SH 浦发银行      15.4644    15.6675    15.7912    14.9345
2016-01-11 600000.SH 浦发银行      15.4467    15.1994    15.4114    14.9786
           收盘价(元) 成交量(股) 成交金额(元) 涨跌(元) 涨跌幅(%) 均价(元)
2016-01-04    15.7205   42240610    754425783  -0.4151   -2.5725  17.8602
2016-01-05    15.8618   58054793   1034181474   0.1413    0.8989  17.8139
2016-01-06    15.9855   46772653    838667398   0.1236    0.7795  17.9307
2016-01-07    15.4644   11350479    199502702  -0.5211   -3.2597  17.5766
2016-01-08    15.4467   71918296   1262105060  -0.0177   -0.1142  17.5492
2016-01-11    15.0581   90177135   1550155933  -0.3886   -2.5157  17.1901
           换手率(%) A股流通市值(元)   总市值(元) A股流通股本(股) 市盈率
2016-01-04    0.2264   332031791187 332031791187     18653471415 6.5614
2016-01-05    0.3112   335016346613 335016346613     18653471415 6.6204
2016-01-06    0.2507   337627832612 337627832612     18653471415 6.6720
2016-01-07    0.0608   326622284477 326622284477     18653471415 6.4545
2016-01-08    0.3855   326249215048 326249215048     18653471415 6.4471
2016-01-11    0.4834   318041687626 318041687626     18653471415 6.2849
```

3.20 题目 70（数据计算）：对收盘价做步长为 5 的滑动平均

难度：★★★

代码及运行结果：

```r
library(slider)

df |>
  mutate(avg_5 = slide_dbl(`收盘价（元）`, mean, na.rm = TRUE,
                           .before = 2, .after = 2)) |>
  select(日期, `收盘价（元）`, avg_5)
```

```
# A tibble: 327 x 3
  日期       `收盘价(元)` avg_5
  <date>            <dbl> <dbl>
1 2016-01-04         15.7  15.9
2 2016-01-05         15.9  15.8
3 2016-01-06         16.0  15.7
4 2016-01-07         15.5  15.6
5 2016-01-08         15.4  15.5
6 2016-01-11         15.1  15.3
# i 321 more rows
```

3.21 题目 71（数据计算）：对收盘价做步长为 5 的滑动求和

难度：★★★

代码及运行结果：

```r
df |>
  mutate(sum_5 = slide_dbl(`收盘价（元）`, sum, na.rm = TRUE,
                           .before = 2, .after = 2)) |>
  select(日期, `收盘价（元）`, sum_5)
```

```
# A tibble: 327 x 3
  日期       `收盘价(元)` sum_5
  <date>            <dbl> <dbl>
1 2016-01-04         15.7  47.6
2 2016-01-05         15.9  63.0
```

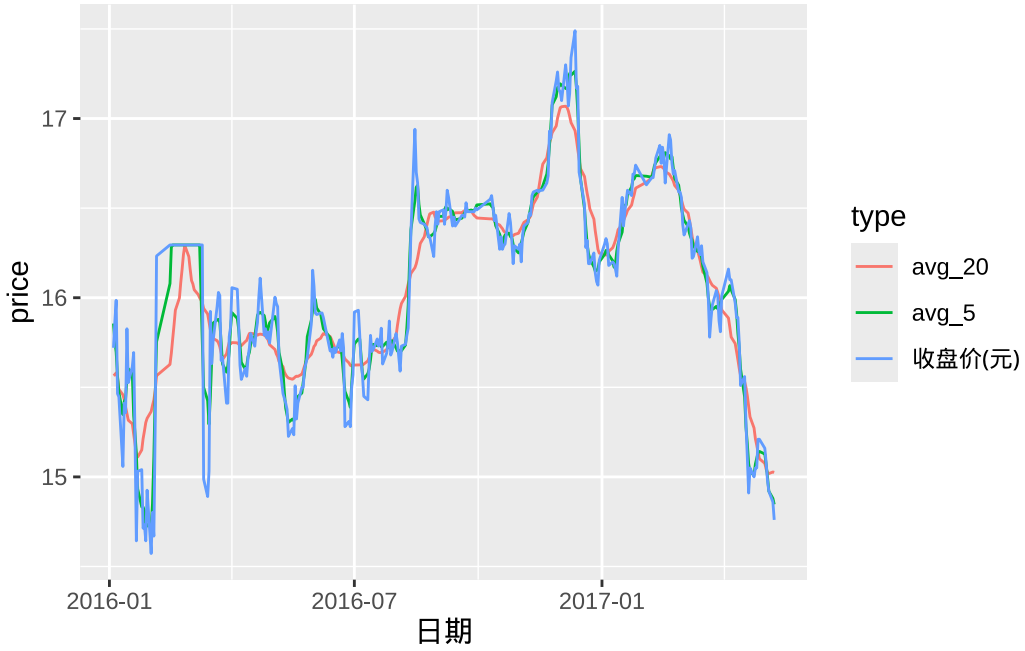
```
3 2016-01-06          16.0  78.5
4 2016-01-07          15.5  77.8
5 2016-01-08          15.4  77.4
6 2016-01-11          15.1  76.7
# i 321 more rows
```

3.22 题目 72（数据可视化）：将收盘价及其 5 日均线、20 日均线绘制在同一个图上

难度：★★★★

代码及运行结果：

```
df |>
  mutate(avg_5 = slide_dbl(`收盘价（元）`, mean, na.rm = TRUE,
                           .before = 2, .after = 2),
  avg_20 = slide_dbl(`收盘价（元）`, mean, na.rm = TRUE,
                     .before = 10, .after = 9)) |>
  pivot_longer(c(`收盘价（元）`, avg_5, avg_20),
               names_to = "type", values_to = "price") |>
  ggplot(aes(日期, price, color = type)) +
    geom_line()
```



3.23 题目 73（数据重采样）：按周为采样规则，计算一周收盘价最大值

难度：★★★★

代码及运行结果：

```
library(tsibble)
weekmax = df |>
  mutate(weeks = yearweek(日期)) |>     # 年-周
  slice_max(`收盘价(元)`, n = 1, by = weeks)
weekmax
```

```
# A tibble: 91 x 19
  代码     简称   日期       `前收盘价(元)` `开盘价(元)` `最高价(元)` `最低价(元)`
  <chr>    <chr> <date>              <dbl>        <dbl>        <dbl>        <dbl>
1 600000~ 浦发银行~ 2016-01-06          15.9         15.8         16.0         15.6
2 600000~ 浦发银行~ 2016-01-14          15.4         15.0         15.9         14.9
3 600000~ 浦发银行~ 2016-01-19          15.6         15.7         15.8         15.5
4 600000~ 浦发银行~ 2016-01-25          15.0         15.0         15.2         14.8
```

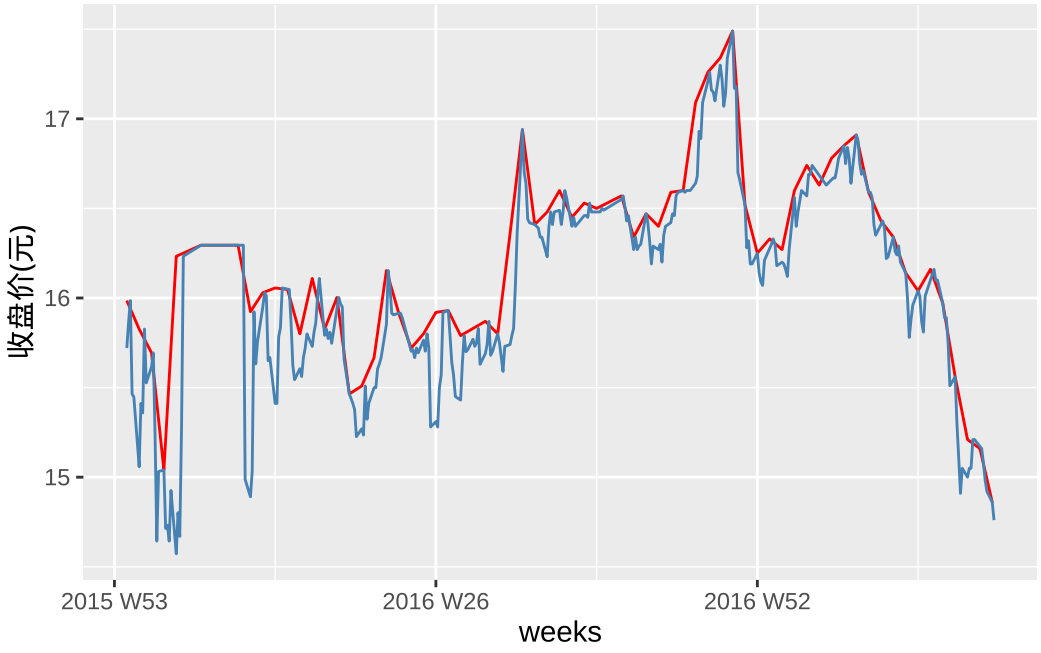
```
5 600000~ 浦发银行~ 2016-02-05          15.3         15.2         16.2         15.1
6 600000~ 浦发银行~ 2016-02-15          16.2         15.9         16.3         15.7
# i 85 more rows
# i 12 more variables: `收盘价(元)` <dbl>, `成交量(股)` <dbl>,
#   `成交金额(元)` <dbl>, `涨跌(元)` <dbl>, `涨跌幅(%)` <dbl>,
#   `均价(元)` <dbl>, `换手率(%)` <dbl>, `A股流通市值(元)` <dbl>,
#   `总市值(元)` <dbl>, `A股流通股本(股)` <dbl>, 市盈率 <dbl>, weeks <week>
```

3.24 题目 74（数据可视化）：绘制重采样数据与原始数据

难度：★★★

代码及运行结果：

```
weekmax |>
  ggplot(aes(weeks, `收盘价 (元)`)) +
  geom_line(color = "red") +
  geom_line(data = df, aes(日期, `收盘价 (元)`), color = "steelblue")
```



3.25 题目 75（数据操作）：将数据往后移动 5 天

难度：★★★

代码及运行结果：

```
df |>
  mutate(across(4:18, \(x) lag(x, 5)))
```

```
# A tibble: 327 x 18
  代码     简称   日期       `前收盘价(元)` `开盘价(元)` `最高价(元)` `最低价(元)`
  <chr>    <chr> <date>              <dbl>        <dbl>        <dbl>        <dbl>
1 600000~ 浦发银行~ 2016-01-04           NA           NA           NA           NA
2 600000~ 浦发银行~ 2016-01-05           NA           NA           NA           NA
3 600000~ 浦发银行~ 2016-01-06           NA           NA           NA           NA
4 600000~ 浦发银行~ 2016-01-07           NA           NA           NA           NA
5 600000~ 浦发银行~ 2016-01-08           NA           NA           NA           NA
6 600000~ 浦发银行~ 2016-01-11         16.1         16.1         16.1         15.5
# i 321 more rows
# i 11 more variables: `收盘价(元)` <dbl>, `成交量(股)` <dbl>,
#   `成交金额(元)` <dbl>, `涨跌(元)` <dbl>, `涨跌幅(%)` <dbl>,
#   `均价(元)` <dbl>, `换手率(%)` <dbl>, `A股流通市值(元)` <dbl>,
#   `总市值(元)` <dbl>, `A股流通股本(股)` <dbl>, 市盈率 <dbl>
```

注：这是批量做后移，单个变量做后移用 `mutate(var = lag(var, 5))` 即可。

3.26 题目 76（数据操作）：将数据往前移动 5 天

难度：★★★

代码及运行结果：

```
df |>
  mutate(across(4:18, \(x) lead(x, 5)))
```

```
# A tibble: 327 x 18
  代码     简称   日期       `前收盘价(元)` `开盘价(元)` `最高价(元)` `最低价(元)`
  <chr>    <chr> <date>              <dbl>        <dbl>        <dbl>        <dbl>
1 600000~ 浦发银行~ 2016-01-04          15.4         15.2         15.4         15.0
2 600000~ 浦发银行~ 2016-01-05          15.1         15.2         15.5         15.1
3 600000~ 浦发银行~ 2016-01-06          15.4         15.5         15.8         15.3
4 600000~ 浦发银行~ 2016-01-07          15.4         15.0         15.9         14.9
5 600000~ 浦发银行~ 2016-01-08          15.8         15.7         16.0         15.5
6 600000~ 浦发银行~ 2016-01-11          15.5         15.4         15.9         15.3
# i 321 more rows
# i 11 more variables: `收盘价(元)` <dbl>, `成交量(股)` <dbl>,
#   `成交金额(元)` <dbl>, `涨跌(元)` <dbl>, `涨跌幅(%)` <dbl>,
#   `均价(元)` <dbl>, `换手率(%)` <dbl>, `A股流通市值(元)` <dbl>,
#   `总市值(元)` <dbl>, `A股流通股本(股)` <dbl>, 市盈率 <dbl>
```

3.27 题目 77 (数据操作)：计算开盘价的累积平均

难度：★★★

代码及运行结果：

```
rlt = df |>
  mutate(累积平均 = cummean(`开盘价 (元)`)) |>
  select(日期, `开盘价 (元)`, 累积平均)
rlt
```

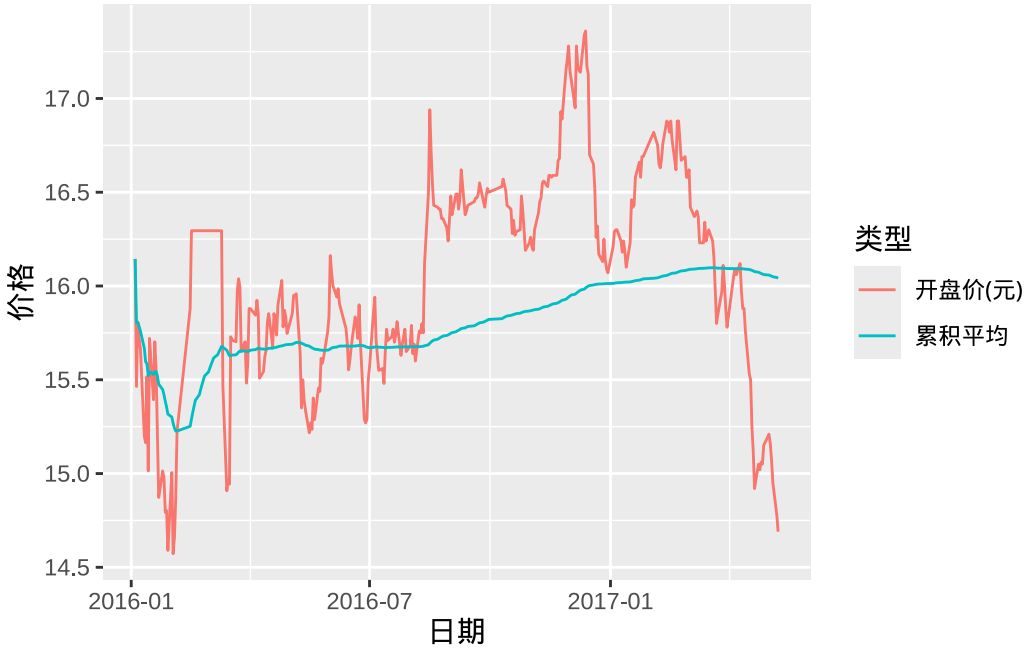
```
# A tibble: 327 x 3
  日期       `开盘价(元)` 累积平均
  <date>            <dbl>    <dbl>
1 2016-01-04         16.1     16.1
2 2016-01-05         15.5     15.8
3 2016-01-06         15.8     15.8
4 2016-01-07         15.7     15.8
5 2016-01-08         15.7     15.8
6 2016-01-11         15.2     15.7
# i 321 more rows
```

3.28 题目 78（数据计算）：绘制开盘价的累积平均与原始数据的折线图

难度：★★★

代码及运行结果：

```
rlt |>
  pivot_longer(-日期, names_to = "类型", values_to = "价格") |>
  ggplot(aes(日期, 价格, color = 类型)) +
    geom_line()
```



3.29 题目 79（数据计算）：计算布林指标

难度：★★★★

代码及运行结果：

```r
boll = df |>
  mutate(avg_20 = slide_dbl(`收盘价（元）`, mean, na.rm = TRUE,
                            .before = 10, .after = 9),
         sd_20 = slide_dbl(`收盘价（元）`, sd, na.rm = TRUE,
                           .before = 10, .after = 9),
         up = avg_20 + 2 * sd_20, down = avg_20 - 2 * sd_20) |>
  select(日期, `收盘价（元）`, avg_20, up, down)

boll |>
  slice_sample(n = 10)
```

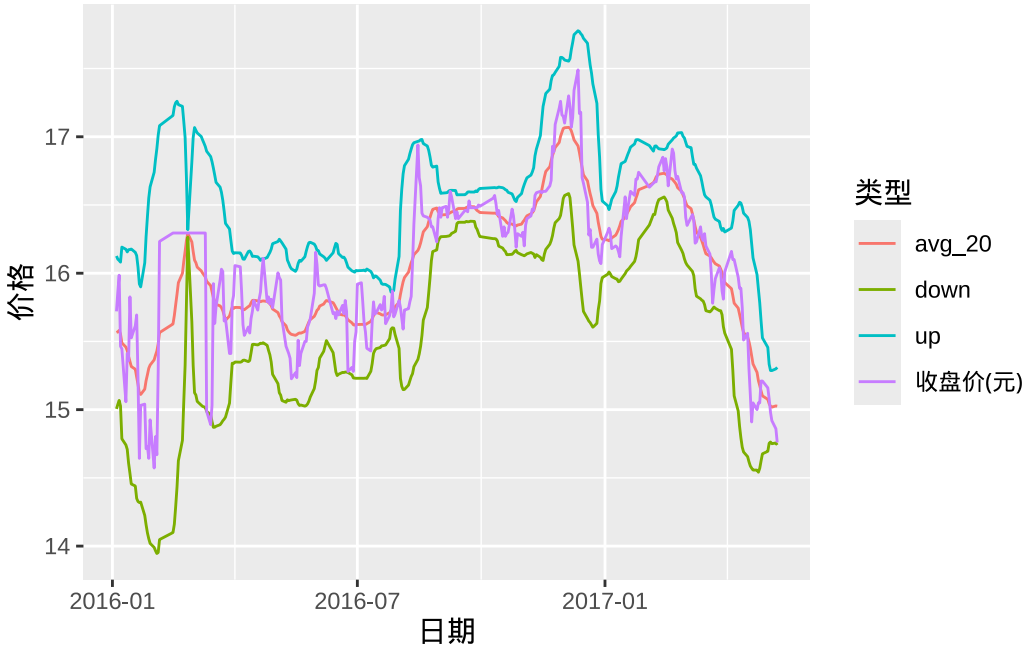
```
# A tibble: 10 x 5
   日期       `收盘价(元)` avg_20    up  down
   <date>            <dbl>  <dbl> <dbl> <dbl>
 1 2017-02-23         16.7   16.7  17.0  16.3
 2 2016-11-11         16.6   16.5  16.9  16.1
 3 2016-04-26         15.8   15.8  16.1  15.4
 4 2016-05-23         15.5   15.6  16.1  15.0
 5 2017-03-15         16.2   16.2  16.6  15.8
 6 2016-02-19         16.3   15.9  17.2  14.6
 7 2016-11-23         16.9   16.9  17.4  16.3
 8 2016-09-19         16.5   16.5  16.6  16.4
 9 2016-06-23         15.7   15.7  16.1  15.3
10 2016-05-25         15.6   15.6  16.2  15.0
```

3.30 题目 80（数据可视化）：绘制布林曲线

难度：★★★

代码及运行结果：

```r
boll |>
  pivot_longer(-日期, names_to = "类型", values_to = "价格") |>
  ggplot(aes(日期, 价格, color = 类型)) +
    geom_line()
```



4 Part VI 数据生成

4.1 题目 81（加载查看包）：加载并查看 tidyverse 包版本

难度：⋆

代码及运行结果：

```
# 得是首次加载
library(tidyverse)
```

4.2 题目 82（生成随机数）：生成 20 个 0~100 的随机数，创建数据框

难度：⋆

代码及运行结果：

```r
set.seed(123)
df1 = tibble(nums = sample(100, 20))
df1
```

```
# A tibble: 20 x 1
   nums
  <int>
1    31
2    79
3    51
4    14
5    67
6    42
# i 14 more rows
```

4.3 题目 83（生成等差数）：生成 20 个 0~100 固定步长的数，创建数据框

难度：⋆

代码及运行结果：

```r
df2 = tibble(nums = seq(0, 99, by = 5))
df2
```

```
# A tibble: 20 x 1
   nums
  <dbl>
1     0
2     5
3    10
4    15
5    20
6    25
# i 14 more rows
```

4.4 题目 84（生成指定分布随机数）：生成 20 个标准正态分布的随机数，创建数据框

难度：★

代码及运行结果：

```
set.seed(123)
df3 = tibble(nums = rnorm(20, 0, 1))
df3
```

```
# A tibble: 20 x 1
     nums
    <dbl>
1 -0.560
2 -0.230
3  1.56
4  0.0705
5  0.129
6  1.72
# i 14 more rows
```

4.5 题目 85（合并数据）：将 df1，df2，df3 按行合并为新数据框

难度：★

代码及运行结果：

```
bind_rows(df1, df2, df3, .id = "source")
```

```
# A tibble: 60 x 2
  source  nums
  <chr>  <dbl>
1 1         31
2 1         79
3 1         51
4 1         14
```

```
5 1         67
6 1         42
# i 54 more rows
```

4.6 题目 86（合并数据）：将 df1，df2，df3 按列合并为新数据框

难度：⋆

代码及运行结果：

```
df = bind_cols(df1, df2, df3)
df
```

```
# A tibble: 20 x 3
  nums...1 nums...2 nums...3
     <int>    <dbl>    <dbl>
1       31        0  -0.560
2       79        5  -0.230
3       51       10   1.56
4       14       15   0.0705
5       67       20   0.129
6       42       25   1.72
# i 14 more rows
```

4.7 题目 87（查看数据）：查看 df 所有数据的最小值、25% 分位数、中位数、75% 分位数、最大值

难度：⋆⋆

代码及运行结果：

```
unlist(df) |>
  summary()
```

```
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
-1.9666  0.4886 25.5000 33.3305 61.2500 97.0000
```

4.8 题目 88（修改列名）：修改列名为 col1，col2，col3

难度：⋆

代码及运行结果：

```
df = df |>
  set_names(str_c("col", 1:3))
df
```

```
# A tibble: 20 x 3
   col1  col2    col3
  <int> <dbl>   <dbl>
1    31     0 -0.560
2    79     5 -0.230
3    51    10  1.56
4    14    15  0.0705
5    67    20  0.129
6    42    25  1.72
# i 14 more rows
```

注：若只修改个别列名，用 rename(newname = oldname).

4.9 题目 89（数据操作）：提取在第 1 列中而不在第 2 列中的数

难度：⋆⋆

代码及运行结果：

```
setdiff(df$col1, df$col2)
```

```
 [1] 31 79 51 14 67 42 43 97 69 57  9 72 26  7 87 36
```

4.10 题目 90（数据操作）：提取在第 1 列和第 2 列出现频率最高的 3 个数值

难度：⋆⋆⋆

代码及运行结果：

```
tibble(nums = c(df$col1, df$col2)) |>
  count(nums, sort = TRUE) |>
  slice(1:3)
```

```
# A tibble: 3 x 2
   nums     n
  <dbl> <int>
1    25     2
2    50     2
3    90     2
```

4.11 题目 91（数据操作）：提取第 1 列可以整除 5 的数的位置

难度：⋆⋆

代码及运行结果：

```
which(df$col1 %% 5 == 0)
```

```
[1]  7 10 11 18
```

- 选取满足条件的索引，通常用途还是用来选出满足条件的行，不兜圈子做法：

```
df |>
  filter(col1 %% 5 == 0)
```

```
# A tibble: 4 x 3
   col1  col2   col3
  <int> <dbl>  <dbl>
1    50    30  0.461
```

```
2    25    45 -0.446
3    90    50  1.22
4    95    85 -1.97
```

4.12 题目 92（数据计算）：计算第 1 列的 1 阶差分

难度：★★

代码及运行结果：

```
df |>
  mutate(diff1 = col1 - lag(col1))
```

```
# A tibble: 20 x 4
   col1  col2    col3 diff1
  <int> <dbl>   <dbl> <int>
1    31     0 -0.560     NA
2    79     5 -0.230     48
3    51    10  1.56     -28
4    14    15  0.0705   -37
5    67    20  0.129     53
6    42    25  1.72     -25
# i 14 more rows
```

注：若只是要数值，用 `diff(df$col1)` 即可。

4.13 题目 93（数据操作）：将 col1，col2，col3 三列顺序颠倒

难度：★★

代码及运行结果：

```
df |>
  select(last_col():1)
```

```
# A tibble: 20 x 3
     col3  col2  col1
    <dbl> <dbl> <int>
1 -0.560      0    31
2 -0.230      5    79
3  1.56      10    51
4  0.0705    15    14
5  0.129     20    67
6  1.72      25    42
# i 14 more rows
```

注：更灵活的调整列序，可以用 `relocate()` 函数。

4.14 题目 94（数据操作）：提取第一列位置在 1,10,15 的数

难度：⋆

代码及运行结果：

```
df[c(1,10,15), 1]
```

```
# A tibble: 3 x 1
   col1
  <int>
1    31
2    25
3    72
```

4.15 题目 95（数据操作）：查找第一列的局部最大值位置

难度：⋆⋆⋆

代码及运行结果：

```r
rlt = df |>
  mutate(diff = sign(col1 - lag(col1)) + sign(col1 - lead(col1)))
which(rlt$diff == 2)
```

```
[1]  2  5  7  9 11 15 18
```

- 不兜圈子做法：

```r
df |>
  mutate(diff = sign(col1 - lag(col1)) + sign(col1 - lead(col1))) |>
  filter(diff == 2)
```

```
# A tibble: 7 x 4
   col1  col2   col3  diff
  <int> <dbl>  <dbl> <dbl>
1    79     5 -0.230     2
2    67    20  0.129     2
3    50    30  0.461     2
4    97    40 -0.687     2
5    90    50  1.22      2
6    72    70 -0.556     2
7    95    85 -1.97      2
```

4.16 题目 96（数据计算）：按行计算 df 每一行的均值

难度：★★

代码及运行结果：

```r
rowMeans(df)    # 或者 apply(df, 1, mean)
```

```
 [1] 10.146508 27.923274 20.852903  9.690169 29.043096 22.905022 26.820305
 [8] 25.578313 45.437716 23.184779 47.074694 41.453271 39.133590 24.703561
[15] 47.148053 34.262304 29.165950 59.344461 59.233785 43.509070
```

```r
# 或者
df |>
  mutate(row_avg = apply(pick(1:3), 1, mean))
```

```
# A tibble: 20 x 4
   col1  col2    col3 row_avg
  <int> <dbl>   <dbl>   <dbl>
1    31     0 -0.560    10.1
2    79     5 -0.230    27.9
3    51    10  1.56     20.9
4    14    15  0.0705    9.69
5    67    20  0.129    29.0
6    42    25  1.72     22.9
# i 14 more rows
```

```r
# 或者
df |>
  mutate(row_avg = pmap_dbl(pick(1:3), \(...) mean(c(...))))
```

```
# A tibble: 20 x 4
   col1  col2    col3 row_avg
  <int> <dbl>   <dbl>   <dbl>
1    31     0 -0.560    10.1
2    79     5 -0.230    27.9
3    51    10  1.56     20.9
4    14    15  0.0705    9.69
5    67    20  0.129    29.0
6    42    25  1.72     22.9
# i 14 more rows
```

4.17 题目 97（数据计算）：对第 2 列计算步长为 3 的移动平均值

*难度：**★★★

代码及运行结果：

```
df |>
  mutate(avg_3 = slide_dbl(col2, mean, .before = 1, .after = 1))
```

```
# A tibble: 20 x 4
   col1  col2    col3 avg_3
  <int> <dbl>   <dbl> <dbl>
1    31     0 -0.560    2.5
2    79     5 -0.230    5
3    51    10  1.56    10
4    14    15  0.0705  15
5    67    20  0.129   20
6    42    25  1.72    25
# i 14 more rows
```

4.18 题目 98（数据计算）：按第 3 列值的大小升序排列

难度：★★

代码及运行结果：

```
df |>
  arrange(col3)
```

```
# A tibble: 20 x 3
   col1  col2   col3
  <int> <dbl>  <dbl>
1    95    85 -1.97
2    43    35 -1.27
3    97    40 -0.687
4    31     0 -0.560
5    72    70 -0.556
6    36    95 -0.473
# i 14 more rows
```

4.19 题目 99（数据操作）：按第 1 列大于 50 的数修改为”高”

难度：★★

代码及运行结果：

```r
df |>
  mutate(col1 = ifelse(col1 > 50, "高", col1))
```

```
# A tibble: 20 x 3
  col1   col2    col3
  <chr> <dbl>   <dbl>
1 31        0 -0.560
2 高        5 -0.230
3 高       10  1.56
4 14       15  0.0705
5 高       20  0.129
6 42       25  1.72
# i 14 more rows
```

4.20 题目 100（数据计算）：计算第 1 列与第 2 列的欧氏距离

难度：★★★

代码及运行结果：

```r
df |>
  summarise(d = (col1 - col2)^2 |> sum() |> sqrt())
```

```
# A tibble: 1 x 1
      d
  <dbl>
1  176.
```

5 Part V 高级

5.1 题目 101（数据读取）：从 csv 文件中读取指定数据：读取前 10 行，positionName 和 salary 列

难度：★★

代码及运行结果：

```
read_csv("data/数据 1_101-120 涉及.csv", n_max = 10,
         col_select = c(positionName, salary))
```

```
# A tibble: 10 x 2
   positionName   salary
   <chr>           <dbl>
 1 数据分析        37500
 2 数据建模        15000
 3 数据分析         3500
 4 数据分析        45000
 5 数据分析        30000
 6 数据分析        50000
 7 数据分析        30000
 8 数据建模工程师  35000
 9 数据分析专家    60000
10 数据分析师      40000
```

5.2 题目 102（数据读取）：从 csv 文件中读取数据，将薪资大于 10000 的改为”高”

难度：★★

代码及运行结果：

```
df = read_csv("data/数据 2_101-120 涉及.csv") |>
  mutate(薪资水平 = if_else(薪资水平 > 10000, "高", "低"))
```

5.3 题目 103（数据操作）：从 df 中对薪资水平每隔 20 行进行抽样

难度：★★

代码及运行结果：

```
df |>
  slice(seq(1, n(), by = 20))
```

```
# A tibble: 58 x 2
  学历要求 薪资水平
  <chr>    <chr>
1 本科     高
2 本科     高
3 本科     高
4 本科     高
5 本科     高
6 本科     高
# i 52 more rows
```

5.4 题目 104（数据操作）：取消使用科学记数法

难度：★★

代码及运行结果：

```
library(scales)
set.seed(123)
df = tibble(val = runif(10) ^ 10)
# 三位小数
df |>
  mutate(val = number(val, accuracy = 0.001))
```

```
# A tibble: 10 x 1
   val
```

```
   <chr>
 1 0.000
 2 0.093
 3 0.000
 4 0.288
 5 0.541
 6 0.000
 7 0.002
 8 0.320
 9 0.003
10 0.000
```

```
# 科学记数法
df |>
  mutate(val = scientific(val, 2))
```

```
# A tibble: 10 x 1
   val
   <chr>
 1 3.9e-06
 2 9.3e-02
 3 1.3e-04
 4 2.9e-01
 5 5.4e-01
 6 3.9e-14
 7 1.7e-03
 8 3.2e-01
 9 2.6e-03
10 3.9e-04
```

5.5 题目 105（数据操作）：将上一题的数据转换为百分数

难度：★★★

代码及运行结果：

```r
df |>
  mutate(val = percent(val, 0.01))
```

```
# A tibble: 10 x 1
   val
   <chr>
 1 0.00%
 2 9.27%
 3 0.01%
 4 28.82%
 5 54.13%
 6 0.00%
 7 0.17%
 8 32.04%
 9 0.26%
10 0.04%
```

5.6 题目 106（数据操作）：查找上一题数据中第 3 大值的行号

难度：★★★

代码及运行结果：

```r
order(df$val, decreasing = TRUE)[3]
```

```
[1] 4
```

- 不兜圈子做法：

```r
df |>
  arrange(-val) |>
  slice(3)
```

```
# A tibble: 1 x 1
    val
  <dbl>
1 0.288
```

5.7 题目 107（数据操作）：反转 df 的行

难度：★★

代码及运行结果：

```
df |>
  slice(rev(1:n()))   # 或者 df[nrow(df):1,]
```

```
# A tibble: 10 x 1
        val
      <dbl>
 1 3.94e- 4
 2 2.60e- 3
 3 3.20e- 1
 4 1.69e- 3
 5 3.85e-14
 6 5.41e- 1
 7 2.88e- 1
 8 1.31e- 4
 9 9.27e- 2
10 3.87e- 6
```

5.8 题目 108（数据连接：全连接）：根据多列匹配合并数据，保留 df1 和 df2 的观测

难度：★★

代码及运行结果：

```r
df1 = tibble(
  key1 = c("K0","K0","K1","K2"),
  key2 = c("K0","K1","K0","K1"),
  A = str_c('A', 0:3),
  B = str_c('B', 0:3))
df1
```

```
# A tibble: 4 x 4
  key1  key2  A     B
  <chr> <chr> <chr> <chr>
1 K0    K0    A0    B0
2 K0    K1    A1    B1
3 K1    K0    A2    B2
4 K2    K1    A3    B3
```

```r
df2 = tibble(
  key1 = c("K0","K1","K1","K2"),
  key2 = str_c("K", rep(0,4)),
  C = str_c('C', 0:3),
  D = str_c('D', 0:3))
df2
```

```
# A tibble: 4 x 4
  key1  key2  C     D
  <chr> <chr> <chr> <chr>
1 K0    K0    C0    D0
2 K1    K0    C1    D1
3 K1    K0    C2    D2
4 K2    K0    C3    D3
```

```r
df1 |>
  full_join(df2, by = c("key1", "key2"))
```

```
# A tibble: 6 x 6
```

```
  key1  key2  A     B     C     D
  <chr> <chr> <chr> <chr> <chr> <chr>
1 K0    K0    A0    B0    C0    D0
2 K0    K1    A1    B1    <NA>  <NA>
3 K1    K0    A2    B2    C1    D1
4 K1    K0    A2    B2    C2    D2
5 K2    K1    A3    B3    <NA>  <NA>
6 K2    K0    <NA>  <NA>  C3    D3
```

5.9 题目 109（数据连接：左连接）：根据多列匹配合并数据，只保留 df1 的观测

难度：★★

代码及运行结果：

```
df1 |>
  left_join(df2, by = c("key1", "key2"))
```

```
# A tibble: 5 x 6
  key1  key2  A     B     C     D
  <chr> <chr> <chr> <chr> <chr> <chr>
1 K0    K0    A0    B0    C0    D0
2 K0    K1    A1    B1    <NA>  <NA>
3 K1    K0    A2    B2    C1    D1
4 K1    K0    A2    B2    C2    D2
5 K2    K1    A3    B3    <NA>  <NA>
```

注：dplyr 包还提供了右连接：right_join()，内连接：inner_join()，以及用于过滤的连接：半连接：semi_join()，反连接：anti_join().

5.10 题目 110（数据处理）：再次读取数据 1 并显示所有列

难度：★★

代码及运行结果：

```
df = read_csv("data/数据 1_101-120 涉及.csv")
glimpse(df)
```

```
Rows: 105
Columns: 53
$ positionId            <dbl> 6802721, 5204912, 6877668, 6496141, 6467417, 688~
$ positionName          <chr> "数据分析", "数据建模", "数据分析", "数据分析", "数据
分析", "数据分析", ~
$ companyId             <dbl> 475770, 50735, 100125, 26564, 29211, 94826, 3487~
$ companyLogo           <chr> "i/image2/M01/B7/3E/CgoB5lwPfEaAdn8WAABWQ0Jgl5s3~
$ companySize           <chr> "50-150人", "150-500人", "2000人以上", "500-
2000人", "~
$ industryField         <chr> "移动互联网,电商", "电商", "移动互联网,企业服务", "电
商", "物流丨运输", "~
$ financeStage          <chr> "A轮", "B轮", "上市公司", "D轮及以上", "上市公
司", "B轮", "A轮",~
$ companyLabelList      <chr> "['绩效奖金', '带薪年假', '定期体检', '弹性工作']", "['
年终奖金', '做~
$ firstType             <chr> "产品|需求|项目类", "开发|测试|运维类", "产品|需求|项
目类", "开发|测试|运维~
$ secondType            <chr> "数据分析", "数据开发", "数据分析", "数据开发", "数据
分析", "数据分析", ~
$ thirdType             <chr> "数据分析", "建模", "数据分析", "数据分析", "数据分
析", "数据分析", "数~
$ skillLables           <chr> "['SQL', '数据库', '数据运营', 'BI']", "['算法', '数
据架构']"~
$ positionLables        <chr> "['电商', '社交', 'SQL', '数据库', '数据运营', 'BI']", "['
算~
$ industryLables        <chr> "['电商', '社交', 'SQL', '数据库', '数据运营', 'BI']", "[]"~
$ createTime            <chr> "2020/3/16 11:00", "2020/3/16 11:08", "2020/3/16~
$ formatCreateTime      <chr> "11:00发布", "11:08发布", "10:33发布", "10:10
发布", "09:~
$ district              <chr> "余杭区", "滨江区", "江干区", "江干区", "余杭区", "
余杭区", "萧山区",~
```

```
$ businessZones         <chr> "['仓前']", "['西兴', '长河']", "['四季青', '钱江
新城']", NA,~
$ salary                <dbl> 37500, 15000, 3500, 45000, 30000, 50000, 30000, ~
$ workYear              <chr> "1-3年", "3-5年", "1-3年", "3-5年", "3-5年", "1-
3年", ~
$ jobNature             <chr> "全职", "全职", "全职", "全职", "全职", "全职", "全
职", "全职", ~
$ education             <chr> "本科", "本科", "本科", "本科", "大专", "本科", "本
科", "不限", ~
$ positionAdvantage     <chr> "五险一金、弹性工作、带薪年假、年度体检", "六险一金,定期
体检,丰厚年终", "五险一金 周~
$ imState               <chr> "today", "disabled", "today", "threeDays", "disa~
$ lastLogin             <chr> "2020/3/16 11:00", "2020/3/16 11:08", "2020/3/16~
$ publisherId           <dbl> 12022406, 5491688, 5322583, 9814560, 6392394, 11~
$ approve               <dbl> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, ~
$ subwayline            <chr> NA, NA, "4号线", "1号线", NA, NA, NA, "2号线", NA, "2
号~
$ stationname           <chr> NA, NA, "江锦路", "文泽路", NA, NA, NA, "丰潭
路", NA, "丰潭~
$ linestaion            <chr> NA, NA, "4号线_城星路;4号线_市民中心;4号线_江锦
路", "1号线_文泽路", N~
$ latitude              <dbl> 30.27842, 30.18804, 30.24152, 30.29940, 30.28295~
$ longitude             <dbl> 120.0059, 120.2012, 120.2125, 120.3503, 120.0098~
$ hitags                <chr> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, ~
$ resumeProcessRate     <dbl> 50, 23, 11, 100, 20, 16, 100, 1, 83, 1, 83, 0, 1~
$ resumeProcessDay      <dbl> 1, 1, 4, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, ~
$ score                 <dbl> 233, 176, 80, 68, 66, 66, 65, 47, 24, 18, 17, 17~
$ newScore              <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ~
$ matchScore            <dbl> 15.1018750, 32.5594140, 14.9723570, 12.8741530, ~
$ matchScoreExplain     <lgl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, ~
$ query                 <lgl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, ~
$ explain               <lgl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, ~
$ isSchoolJob           <dbl> 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, ~
$ adWord                <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ~
$ plus                  <lgl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, ~
```

```
$ pcShow                <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ~
$ appShow               <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ~
$ deliver               <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ~
$ gradeDescription      <lgl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, ~
$ promotionScoreExplain <lgl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, ~
$ isHotHire             <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ~
$ count                 <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ~
$ aggregatePositionIds  <chr> "[]", "[]", "[]", "[]", "[]", "[]", "[]", "[]", ~
$ famousCompany         <lgl> FALSE, FALSE, FALSE, TRUE, TRUE, FALSE, FALSE, T~
```

5.11 题目 111（数据操作）：查找 secondType 与 thirdType 值相等的行号

难度：★★

代码及运行结果：

```
which(df$secondType == df$thirdType)
```

```
 [1]   1   3   5   6   7  11  15  24  26  28  29  30  31  34  38  39  40  41  42
[20]  49  50  53  54  56  58  62  66  67  68  72  74  75  76  80  81  83  86  89
[39]  90  92  97 101
```

- 不兜圈子：

```
df |>
  filter(secondType == thirdType)
```

```
# A tibble: 42 x 53
  positionId positionName companyId companyLogo        companySize industryField
       <dbl> <chr>            <dbl> <chr>              <chr>       <chr>
1    6802721 数据分析        475770 i/image2/M01/B7/3~ 50-150人    移动互联网,电商
~~
2    6877668 数据分析        100125 image2/M00/0C/57/~ 2000人以上  移动互联网,企业
服务~~
3    6467417 数据分析         29211 i/image2/M01/77/B~ 2000人以上  物流丨运输
```

```
4    6882347 数据分析          94826 image2/M00/04/12/~ 50-150人    移动互联网,社交
~~
5    6841659 数据分析         348784 i/image2/M01/AC/0~ 50-150人    移动互联网,电商
~~
6    6804629 数据分析师        34132 i/image2/M01/F8/D~ 150-500人   数据服务,广告
营销~~
# i 36 more rows
# i 47 more variables: financeStage <chr>, companyLabelList <chr>,
#   firstType <chr>, secondType <chr>, thirdType <chr>, skillLables <chr>,
#   positionLables <chr>, industryLables <chr>, createTime <chr>,
#   formatCreateTime <chr>, district <chr>, businessZones <chr>, salary <dbl>,
#   workYear <chr>, jobNature <chr>, education <chr>, positionAdvantage <chr>,
#   imState <chr>, lastLogin <chr>, publisherId <dbl>, approve <dbl>, ...
```

5.12 题目 112（数据操作）：查找薪资大于平均薪资的第 3 个数据

难度：★★★

代码及运行结果：

```
df |>
  filter(salary > mean(salary)) |>
  slice(3)
```

```
# A tibble: 1 x 53
  positionId positionName companyId companyLogo        companySize industryField
       <dbl> <chr>            <dbl> <chr>              <chr>       <chr>
1    6882347 数据分析         94826 image2/M00/04/12/~ 50-150人    移动互联网,社交
~~
# i 47 more variables: financeStage <chr>, companyLabelList <chr>,
#   firstType <chr>, secondType <chr>, thirdType <chr>, skillLables <chr>,
#   positionLables <chr>, industryLables <chr>, createTime <chr>,
#   formatCreateTime <chr>, district <chr>, businessZones <chr>, salary <dbl>,
#   workYear <chr>, jobNature <chr>, education <chr>, positionAdvantage <chr>,
#   imState <chr>, lastLogin <chr>, publisherId <dbl>, approve <dbl>,
#   subwayline <chr>, stationname <chr>, linestaion <chr>, latitude <dbl>, ...
```

5.13 题目 113（数据操作）：将上一题数据的 salary 列开根号

难度：★★

代码及运行结果：

```
df |>
  mutate(salary_sqrt = sqrt(salary), .before = 3)
```

```
# A tibble: 105 x 54
  positionId positionName salary_sqrt companyId companyLogo          companySize
       <dbl> <chr>              <dbl>     <dbl> <chr>                <chr>
1    6802721 数据分析           194.     475770 i/image2/M01/B7/3E/~ 50-150人
2    5204912 数据建模           122.      50735 image1/M00/00/85/Cg~ 150-500人
3    6877668 数据分析            59.2    100125 image2/M00/0C/57/Cg~ 2000人以上
4    6496141 数据分析           212.      26564 i/image2/M01/F7/3F/~ 500-2000人
5    6467417 数据分析           173.      29211 i/image2/M01/77/B8/~ 2000人以上
6    6882347 数据分析           224.      94826 image2/M00/04/12/Cg~ 50-150人
# i 99 more rows
# i 48 more variables: industryField <chr>, financeStage <chr>,
#   companyLabelList <chr>, firstType <chr>, secondType <chr>, thirdType <chr>,
#   skillLables <chr>, positionLables <chr>, industryLables <chr>,
#   createTime <chr>, formatCreateTime <chr>, district <chr>,
#   businessZones <chr>, salary <dbl>, workYear <chr>, jobNature <chr>,
#   education <chr>, positionAdvantage <chr>, imState <chr>, ...
```

5.14 题目 114（数据操作）：将上一题数据的 linestation 列按_拆分

难度：★★★

代码及运行结果：

```
df |>
  separate(linestaion, into = c("line", "station"), sep = "_",
           remove = FALSE) |>
  select(linestaion, line, station)
```

```
# A tibble: 105 x 3
  linestaion                                  line  station
  <chr>                                       <chr> <chr>
1 <NA>                                        <NA>  <NA>
2 <NA>                                        <NA>  <NA>
3 4号线_城星路;4号线_市民中心;4号线_江锦路 4号线 城星路;4号线
4 1号线_文泽路                               1号线 文泽路
5 <NA>                                        <NA>  <NA>
6 <NA>                                        <NA>  <NA>
# i 99 more rows
```

注：正常需要先按 “;” 拆分，再分别按 “-” 拆分。

5.15 题目 115（数据查看）：查看上一题数据一共有多少列

难度：⋆

代码及运行结果：

```
ncol(df)
```

```
[1] 53
```

5.16 题目 116（数据操作）：提取 industryField 列以”数据”开头的行

难度：⋆⋆

代码及运行结果：

```
df |>
  filter(str_detect(industryField, "^数据"))
```

```
# A tibble: 15 x 53
  positionId positionName   companyId companyLogo      companySize industryField
       <dbl> <chr>              <dbl> <chr>            <chr>       <chr>
1    6458372 数据分析专家       34132 i/image2/M01/F8~ 150-500人   数据服务,广告
营销~~
2    6804629 数据分析师         34132 i/image2/M01/F8~ 150-500人   数据服务,广告
营销~~
3    6804489 资深数据分析师     34132 i/image2/M01/F8~ 150-500人   数据服务,广告
营销~~
4    6267370 数据分析专家       31544 image1/M00/00/4~ 150-500人   数据服务
5    6804489 资深数据分析师     34132 i/image2/M01/F8~ 150-500人   数据服务,广告
营销~~
6    6242470 数据分析师         31544 image1/M00/00/4~ 150-500人   数据服务
# i 9 more rows
# i 47 more variables: financeStage <chr>, companyLabelList <chr>,
#   firstType <chr>, secondType <chr>, thirdType <chr>, skillLables <chr>,
#   positionLables <chr>, industryLables <chr>, createTime <chr>,
#   formatCreateTime <chr>, district <chr>, businessZones <chr>, salary <dbl>,
#   workYear <chr>, jobNature <chr>, education <chr>, positionAdvantage <chr>,
#   imState <chr>, lastLogin <chr>, publisherId <dbl>, approve <dbl>, ...
```

5.17 题目 117 (数据分组汇总)：以 salary score 和 positionID 做数据透视表

难度：★★★

代码及运行结果：

```
df |>
  summarise(salary_avg = mean(salary), score_avg = mean(score),
            .by = positionId)
```

```
# A tibble: 95 x 3
  positionId salary_avg score_avg
       <dbl>      <dbl>     <dbl>
1    6802721      37500       233
```

```
2    5204912      15000       176
3    6877668       3500        80
4    6496141      45000        68
5    6467417      30000        66
6    6882347      50000        66
# i 89 more rows
```

5.18 题目 118（数据分组汇总）：同时对 salary、score 两列进行汇总计算

难度：★★★

代码及运行结果：

```
df |>
  summarise(across(c(salary, score),
                   list(sum=sum, mean=mean, min=min),
                   .names = "{.col}_{.fn}"))
```

```
# A tibble: 1 x 6
  salary_sum salary_mean salary_min score_sum score_mean score_min
       <dbl>       <dbl>      <dbl>     <dbl>      <dbl>     <dbl>
1    3331000      31724.       3500      1335       12.7         0
```

注：若要分组再这样汇总，设置.by 参数即可。

5.19 题目 119（数据分组汇总）：同时对不同列进行不同的汇总计算：对 salary 求平均，对 score 求和

难度：★★★

代码及运行结果：

```
df |>
  summarise(salary_avg = mean(salary), score_sum = sum(score))
```

```
# A tibble: 1 x 2
  salary_avg score_sum
       <dbl>     <dbl>
1     31724.      1335
```

注：若要分组再这样汇总，加上.by 参数即可。

5.20 题目 120（数据分组汇总）：计算并提取平均薪资最高的区

难度：★★★★

代码及运行结果：

```
df |>
  summarise(salary_avg = mean(salary), .by = district) |>
  slice_max(salary_avg, n = 1)
```

```
# A tibble: 1 x 2
  district salary_avg
  <chr>         <dbl>
1 萧山区        36250
```